ViPlan: A Benchmark for Visual Planning with Symbolic Predicates and Vision-Language Models

Matteo Merler[1,2*], **Nicola Dainese**[2*], **Minttu Alakuijala**[2], **Giovanni Bonetta**[1], **Pietro Ferrazzi**[1,3], **Yu Tian**[2], **Bernardo Magnini**[1] and **Pekka Marttinen**[2]

[1]Fondazione Bruno Kessler
[2]Department of Computer Science, Aalto University
[3]Department of Mathematics, Università degli Studi di Padova

Abstract

Integrating Large Language Models with symbolic planners is a promising direction for obtaining verifiable and grounded plans, with recent work extending this idea to visual domains using Vision-Language Models (VLMs). However, a rigorous comparison with methods that plan directly with VLMs is missing, due to a lack of visual benchmarks that support symbolic planning. We present **ViPlan**, the first open-source benchmark for comparing VLM-grounded symbolic approaches (VLM-as-grounder) with direct VLM planning methods (VLM-as-planner). ViPlan introduces a series of increasingly challenging tasks in two visual domains: a visual variant of the classic Blocksworld planning problem and a simulated household robotics environment. We find VLM-as-grounder methods to outperform direct VLM planning in Blocksworld (solving 46% of the tasks against 9%), where image grounding is both crucial and accurate. However, in the household robotics tasks, where linguistic knowledge helps, VLM-as-planner methods are greatly superior to VLM-as-grounder approaches (solving 34% of the tasks against 5%), which are hindered by partial observability. Thus, ViPlan domains capture fundamental shortcomings of both planning approaches, which we further diagnose with a qualitative failure analysis. Finally, across methods, we observe no consistent benefit from Chain-of-Thought prompting, suggesting persistent limitations in current VLMs' visual reasoning abilities.

1 Introduction

Automated planning is a fundamental capability for general-purpose agents, consisting of the ability to generate action plans based on their knowledge of the environment they act in. This enables them to make decisions, adapt to dynamic environments and achieve complex goals (Ghallab *et al.*, 2004). The rise in popularity of foundation models (Bommasani *et al.*, 2021), including Large Language Models (LLMs), has prompted many to test their ability for planning (Ahn *et al.*, 2022; Huang *et al.*, 2022, 2023). Others have been more critical, arguing that LLMs remain unreliable and lack the capacity for formal planning (Valmeekam *et al.*, 2023b; Kambhampati *et al.*, 2024) and recommend integrating them with symbolic planners. Such planners rely on formal logic, for example using the Planning Domain Definition Language (PDDL; McDermott *et al.*, 1998), to specify the relevant properties of the world state and how actions affect them in order to solve tasks.

Vision-Language Models (VLMs; Liu *et al.*, 2023b; Li *et al.*, 2023a) have gathered interest for their ability to process visual observations jointly with language. Inspired by LLM approaches to planning, this has led to two complementary uses of VLMs, distinguished by the role of the model (see Figure 1). The first, which we name **VLM-as-planner**, uses the model to directly select actions from visual inputs. The second, **VLM-as-grounder**, instead uses the VLM to ground symbolic representations of the world, such as the boolean variables defined in PDDL, named *predicates*, in perceptual data. The latter usage is particularly important in open-world domains, such as embodied AI tasks, where the state of the world can change in ways that symbolic planners alone cannot handle due to their lack of built-in perceptual grounding.

However, recent evidence has shown that current state-of-the-art VLMs can still struggle with detecting objects (Augustin *et al.*, 2025) and identifying relationships between entities (Tong *et al.*, 2024), crucial abilities for grounding planners to observations. Furthermore, they inherit the same limitations of LLMs in terms of formal planning abilities. As a result, both approaches exhibit distinct failure modes, yet their relative strengths and weaknesses remain poorly understood. To date, no open-source benchmark enables a comparison of the two under matched environments and protocols, with previous studies either focusing on direct VLM planning (Liu *et al.*, 2025), or working with private benchmarks and closed-source models (Zhang *et al.*, 2025), making it difficult to assess when each approach is preferable.

To address this gap, we present **ViPlan**[1], the first open-source visual planning benchmark designed to compare VLM-as-planner and VLM-as-grounder approaches. In summary, our main contributions are:

[*]Equal contribution.
Preprint, under review.

[1]We open-source ViPlan at https://github.com/merlerm/ViPlan.

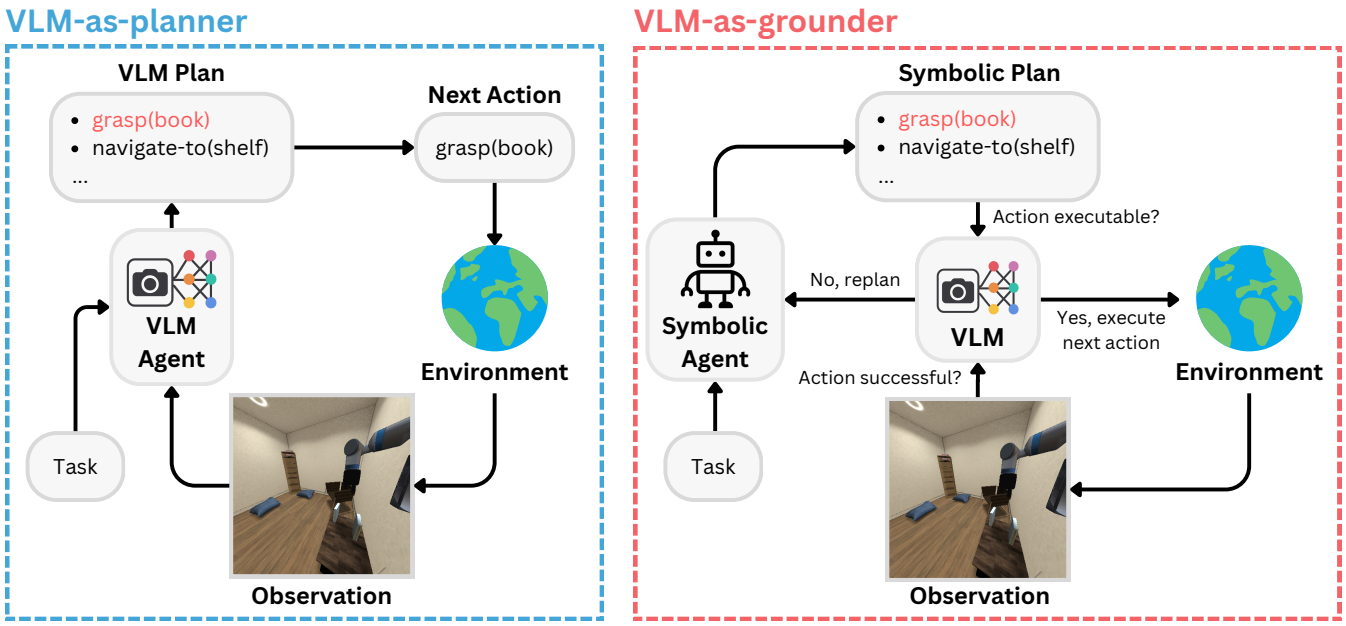


Figure 1: **Planning with VLMs.** Two classes of VLM-based methods for planning a set of actions to reach a goal. **VLM-as-planner** uses the VLM directly to produce a new plan after every action. **VLM-as-grounder** uses the VLM to ground a symbolic agent's plans to the observations from the environment. Grounding takes the form of yes-no question-answering about whether the conditions that make an action executable are met and whether the expected outcomes of the action are realized.

- We develop two domains with complementary reasoning demands: **ViPlan-Blocksworld** (ViPlan-BW), a visual variant of the classic abstract planning problem, and **ViPlan-Household** (ViPlan-HH), a simulated robotics environment featuring navigation and manipulation, each with three increasing difficulty splits of 25 tasks.
- We evaluate 21 popular open-source VLMs with two representative methods, and then select the top five models to benchmark a full suite of eight method implementations (covering both VLM-as-planner and VLM-as-grounder).
- We show a clear trade-off between approaches: VLM-as-grounder consistently outperforms in ViPlan-BW, where precise grounding is crucial and accurate. Conversely, VLM-as-planner is superior in ViPlan-HH, where we hypothesize it exploits linguistic priors to propose plausible actions despite visual ambiguity. This does not apply in ViPlan-BW, where the linguistic context provides no advantage compared to the perceptual input.
- We analyze the impact of Chain-of-Thought prompting (CoT; Wei *et al.*, 2022), showing that it fails to consistently improve performance in visual planning, aligning with recent findings on the limitations of VLM reasoning (Chen *et al.*, 2024; Shiri *et al.*, 2024).
- We identify prevalent failure modes of each approach through a qualitative analysis, distinguishing between grounding hallucinations, planning inconsistencies, and parsing errors.
- We open-source ViPlan and design it to be easily reproducible and extensible with new VLMs, domains and planning methods.

2 Background

We investigate **classical planning problems** (Ghallab *et al.*, 2004). These involve finding a sequence of actions that transitions an agent from an initial state to one that achieves a goal. A classical planning problem is defined as a tuple $\mathcal{P} = \langle \mathcal{D}, \mathcal{I}, \mathcal{G} \rangle$, where $\mathcal{D}$ is the problem **domain**, $\mathcal{I}$ is the **initial state** and $\mathcal{G}$ is the desired **goal state**. The domain $\mathcal{D}$ is further defined as $\mathcal{D} = \langle \mathcal{P}, \mathcal{A} \rangle$, where $\mathcal{P}$ is the set of all boolean **predicates** used to describe the environment's state, and $\mathcal{A}$ is the set of all **actions** the agent can perform. See Figure 2 for a visual example.

Predicates represent properties of the world and can be either **lifted** (parameterized) or **grounded** (instantiated). For example, the lifted predicate `(holding ?m - movable)`[2] becomes grounded as `(holding bowl)` when the argument is instantiated. Only grounded predicates can be evaluated as true or false. The initial state $\mathcal{I}$, goal state $\mathcal{G}$, and generally any state s_t are specified as the set of true grounded predicates at timestep t, with the rest assumed to be false under the closed-world assumption.

Each action in $\mathcal{A}$ is defined by a set of **preconditions** over grounded predicates that must hold in the current state for the action to be applicable, and a set of **effects** that describe how grounded predicates truth values change after the action is executed. Actions, like predicates, can be lifted and then grounded by instantiating their parameters. Actions are typically executed by low-level controllers assumed to perfectly achieve their effects, following the **downward refinement**

[2] `movable` is an example of object type, used to restrict certain actions or predicates to a given object category.

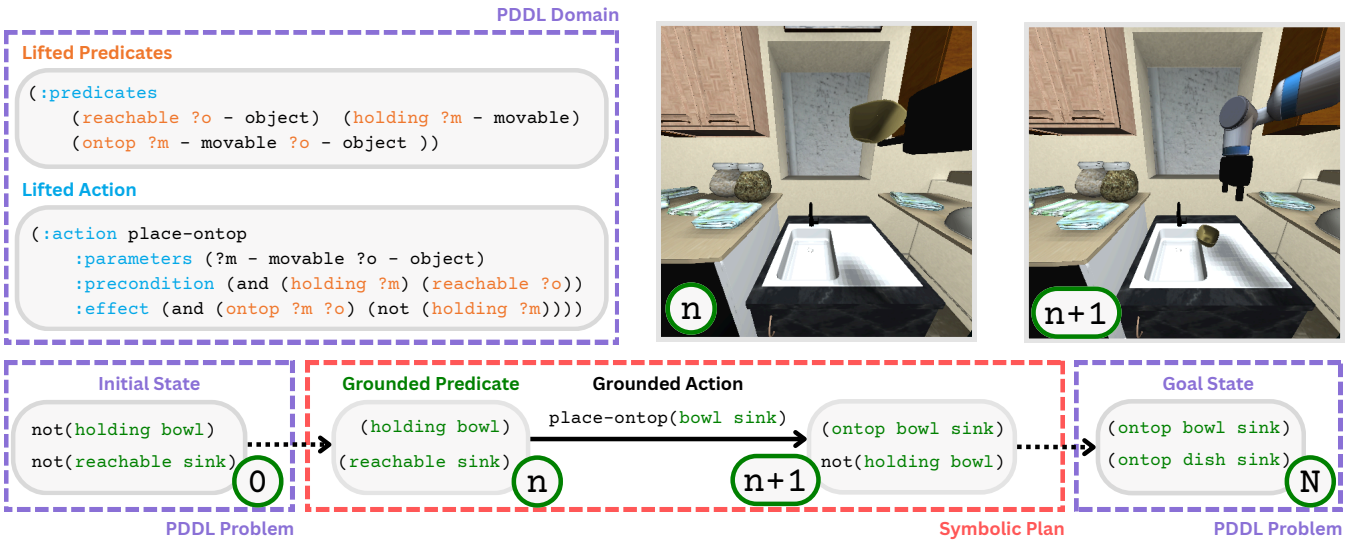


Figure 2: **Example of the basic components for formal planning with PDDL.** A PDDL domain includes the list of possible lifted actions, which are then grounded by a PDDL problem, that provides the initial and goal state. A symbolic planner takes as input the PDDL domain and problem to generate a symbolic plan to reach the desired goal state through a sequence of N action.

property (Bacchus and Yang, 1991) and this allows planners to ignore intermediate states. However, real-world controllers may fail or face changing conditions, and as such grounding is required to identify these failures.

Classical planning problems are typically described using the PDDL formal language (McDermott *et al.*, 1998). A PDDL domain file defines the space of available predicates and actions (along with object types), while a PDDL problem file defines the initial and goal states (in terms of grounded predicate truth values) along with available objects (i.e., specific instances of object types). Example PDDL domain and problem files are provided in Appendix H.

We use the term **problem** to refer to a specific PDDL problem file (i.e., $\langle \mathcal{I}, \mathcal{G} \rangle$ plus object list), and **task** to refer to a concrete instance of that problem (e.g., a specific environment layout using the same domain). A planner takes a domain and problem file and returns a **plan**, which is a list of grounded actions that leads from $\mathcal{I}$ to $\mathcal{G}$.

Note that in classical planning, the planner operates with the symbolic state s_t, which is assumed to be given. We instead study a more realistic scenario, which we refer to as **visual planning**, where only the image x_t of the environment at time t is available, while s_t must be extracted (e.g., with a VLM). This also implies that $\mathcal{I}$ is not known, but must be established from the first image x_0.

3 Related Work

Plan Synthesis with Vision-Language Models. A prominent line of research employs LLMs and VLMs as planners, prompted to directly select actions from a predefined list (Ahn *et al.*, 2022; Huang *et al.*, 2023). LLM-Planner (Song *et al.*, 2023) was among the first to integrate visual feedback through an object detector informing the LLM. KnowNo (Ren *et al.*, 2023) performs multiple-choice QA over the next possible actions, and asks for clarifications when the confidence in the answers is low; however, it uses a vision oracle for perception. LLM-State (Chen *et al.*, 2023a) builds a custom state representation using LLMs that receive a text-based observation coming from an object detector, and then plans based on it, with a similar method presented by Wu *et al.* (2023). DoReMi (Guo *et al.*, 2024) uses an LLM to generate a plan and a set of constraints that hold true during the execution of a specific skill, and then employs a VLM to continuously check if these constraints are broken, and replan if that's the case. ReplanVLM (Mei *et al.*, 2024) uses a VLM to generate a plan and reflects to replan in case of failures. Similarly, ViLa (Hu *et al.*, 2024), which we follow in one implementation of our VLM-as-planner in the benchmark, uses a VLM in a closed-loop fashion by generating a new plan at each step. Crucially, all of these methods build systems upon foundation models without closely investigating the choice of the VLM itself or formalizing a benchmark.

Visual Symbolic Planning. The integration of LLMs and symbolic planning frameworks, such as PDDL, was first proposed by LLM+P (Liu *et al.*, 2023a), with this line of work typically relying on the language model to generate a symbolic domain to be used with classical planners (Huang *et al.*, 2024). VisualPredicator (Liang *et al.*, 2024) uses VLMs to generate PDDL predicates together with a Python implementation, which can use a VLM to query their truth value in the world. Image2PDDL (Dang *et al.*, 2025) generates a PDDL domain and problem starting from images representing the initial and goal states. AHA (Duan *et al.*, 2024) fine-tunes a VLM to reason about failures and then refines VLM-generated symbolic plans based on this feedback. Athalye *et al.* (2024) start from an initial set of predicates and learn more complex predicates and actions through interaction directly from images. VLM-TAMP (Yang *et al.*, 2024) asks the VLM to provide a high-level plan and then reaches these sub-goals

using classical motion planning.

Recently, VLMs have emerged as powerful tools for grounding classical planners with visual perception, which was previously only possible through domain-specific models (Migimatsu and Bohg, 2022), which were trained end-to-end. S3E (Azran *et al.*, 2025) enumerates all possible predicate-object combinations and converts them into natural language questions for a VLM at every step of the plan. The method however is not scalable and the work does not investigate the success rate of planning with the estimated states, which is hard to predict from prediction accuracy alone.

Most relevant to our work, TP-VQA (Zhang *et al.*, 2023) and the later version DKPROMPT (Zhang *et al.*, 2025) use VLMs to monitor the truth values of predicates by asking specific questions based on an action's preconditions and effects. Our implementations of the VLM-as-grounder method class, while independently developed, resemble the DKPROMPT method, as asking yes-no questions about visual predicates is one of the most straightforward ways of interfacing a VLM and a PDDL planner. However, DKPROMPT is tested using solely closed-source models on a private OmniGibson (Li *et al.*, 2024) set of only five tasks, and no code is publicly available at the time of writing. Most importantly, the lack of diversity in environments and of a comparison with strong VLM-as-planner make it hard to assess the full promise of the method. The results on our benchmark, which addresses all the points above, paint a very different picture on how competitive VLM-as-grounder approaches are in home robotics tasks such as ViPlan-HH, showing that VLM-as-planners are clearly superior in our experiments.

Visual Planning Benchmarks. Finally, while benchmarks like VisualAgentBench (Liu *et al.*, 2025) and EmbodiedBench (Yang *et al.*, 2025) evaluate VLM-based agents acting as planners across multiple interactive domains, they are not designed to support symbolic planners and thus cannot evaluate the VLM-as-grounder approach. To the best of our knowledge, no open-source benchmark enables a comparison of VLM-as-planner and VLM-as-grounder methods in visual planning tasks under matched environments and protocols.

For further discussion on works related to benchmarking VLMs for hallucination detection and Visual Question Answering (VQA), we refer the reader to Appendix C.

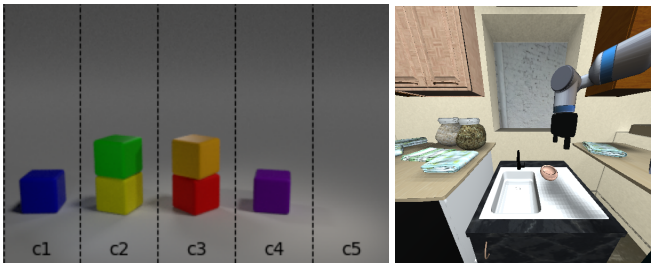
4 The ViPlan Benchmark

This Section describes the two ViPlan domains, as well as the two classes of methods tested, together with the specific method instances. We also introduce the methodology for selecting the VLMs used for our experiments.

4.1 Domains

ViPlan is comprised of two domains: **ViPlan-Blocksworld** (ViPlan-BW) and **ViPlan-Household** (ViPlan-HH), specifically chosen to represent both abstract fully observable, and realistic partially observable environments, which present opposing challenges.

ViPlan-Blocksworld. Blocksworld is a popular classical planning domain (McDermott, 2000) in which the agent must stack blocks on a table in a specific configuration. Our ViPlan-BW domain is written for the photorealistic Blocksworld renderer introduced by Asai (2018). Blocks can only be moved to the top of a column and can only be moved if they do not have other blocks on top. We consider blocks with identical sizes and shapes, but varying in color. ViPlan-BW is composed of 25 distinct procedurally generated problems for three difficulty splits: simple, medium and hard (see Appendix H.1).



(a) ViPlan-BW (b) ViPlan-HH

Figure 3: **Example Domain Visualizations.** We show an example rendering for a state in ViPlan-BW (left) and ViPlan-HH (right).

ViPlan-Household. While ViPlan-BW provides a controlled environment, we are also interested in evaluating planning in a domain closer to real-world embodied AI scenarios. For this, we implement ViPlan-HH on top of iGibson 2.0 (Li *et al.*, 2022), specifically using the Fetch robot. iGibson is an open-source household simulator, where the agent can interact with various objects to complete tasks (e.g., setting the table or cleaning the dishes).

As PDDL deals with high-level actions, we implement each one by bypassing low-level control and instead directly transitioning the environment to one of the action's valid terminal states, saving time and simulation cost. In order to introduce realistic variability, object positions are sampled within allowable regions, a process which also introduces occasional action failures that need to be detected. Due to this randomness, we verify that each task is solvable using an oracle symbolic planner which receives the full symbolic state from the simulator. As in ViPlan-BW, we divide ViPlan-HH in three splits with 25 tasks each. All our original contributions, implementation details, domain and task information can be found at Appendix H.2.

In ViPlan-HH, partial observability means that not all predicates are visible from the agent's current viewpoint, and so we assume the initial state $\mathcal{I}$ is partially known, with non-visible predicates initialized using privileged information from the simulator. This could alternatively be provided through other means in practical settings, such as natural language communication.

Furthermore, some ViPlan-HH tasks include multiple instances of the same object, which look identical and would thus be impossible to disambiguate. In such cases, in order to isolate legitimate grounding/planning errors from ambiguous object detection mistakes, we add labeled bounding boxes with object names (e.g., "book_1", "book_2") only to the duplicate objects. This disambiguates object references, but adds no relational information, thus the VLM must still reason about

actions and object relationships.

4.2 Visual Planning Methods

We consider two classes of methods, **VLM-as-planner** and **VLM-as-grounder**, each implemented with four specific instances, to investigate design choices within each class.

VLM-as-planner. At each timestep, the VLM is provided with an image of the current state, a textual description of the goal and the set of all available actions; additionally, in the case of ViPlan-HH, we provide natural language information about non-visible objects. Depending on the specific method instance, the VLM is then tasked with generating either a full plan in a parsable format (JSON) that satisfies the goal (Plan variant), or the next action (Action variant). We further consider two variants where the VLM is asked to perform a CoT reasoning trace before generating the plan or the action, named respectively Plan + CoT and Action + CoT variants. In the case of the two Plan variants, only the first action of the plan is executed and the VLM is prompted to generate a full plan again at the next step. This kind of closed-loop planning is well-suited for our tasks, as it can naturally recover from action failures.

VLM-as-grounder. The VLM-as-grounder approach uses the model to ground a symbolic planner (specifically, the Fast Downward planner (Helmert, 2006), following the Unified Planning library's (Micheli *et al.*, 2025) implementation).

First, the VLM is tasked with filling in the starting truth values for all possible grounded predicates in the initial state $\mathcal{I}$ (which we call **predicate enumeration**), so that the symbolic planner can generate an initial plan. After, for every action in the plan to be executed, the VLM is first prompted to verify that the preconditions hold, and then to verify that all effects are observed as expected after execution, as shown in Figure 1. We verify each predicate by first converting it to a natural language yes-no question through a pre-defined template (e.g., `(holding bowl)` becomes "Is the agent holding a bowl?") and then prompting the VLM to answer the question based on the current state image x_t. We consider two base variants: Ground, which uses this standard yes-no question-answering, and Ground + CoT, which augments each question with a request for CoT reasoning before the answer.

If, at any point, there is a mismatch between the VLM answers and the values expected by the PDDL action, the state of the world is deemed inconsistent. In this case, the action might have had unintended consequences, and changed other predicates besides the ones specified by its effects. For example, a bowl could be knocked over while moving another one due to a failed low-level execution. Thus, we must re-establish the state through predicate enumeration, checking all combinations of visually verifiable grounded predicates. Afterwards, a new plan is generated, and the agent can continue the task while recovering from action failures.

Memory-augmented variants. Additionally, when the VLM determines that preconditions are not met (preventing an action, correctly or mistakenly deemed invalid) or when an attempted action is found non-executable (revealing that the VLM incorrectly assessed the preconditions as satisfied), this signals a potential grounding error. In such cases, we test variants that augment the VLM's context during the subsequent replanning cycle with **memory** of the grounding issue. We provide the action previously attempted, the precondition questions with their VLM answers, and whether the issue was caught during precondition checking or only discovered upon execution attempt. These memory-augmented variants are Ground + Mem and Ground + Mem + CoT, which combines memory with CoT reasoning.

4.3 Vision-Language Model Selection

VLM Pool. To ensure comprehensive coverage of the current VLM landscape, we evaluate a diverse pool of 21 open-source vision-language models spanning multiple model families and scales. Our model pool includes: AyaVision (8B, 32B), Cosmos-Reason 2 (8B), DeepSeek-VL2 (27B A4.5B), Gemma-3 (12B, 27B), InternVL3 (8B, 78B), InternVL3.5 (8B, 38B, 30B A3B), Mistral-Small-3.1 (24B), Molmo (7B), Llava-OneVision (7B, 72B), Phi-4 Multimodal (5.6B), Qwen2.5-VL (7B, 72B) and Qwen3-VL (8B, 32B, 30B A3B)[3]. Additionally, we include three closed-source models for reference: GPT-5.2, GPT-4.1 and GPT-4.1 Nano. We exclude these models from the selection process in order to keep the benchmark based on open-source models, but still test them to provide valuable context for assessing the gap between open and closed-source VLMs. We report the full model names as found in Hugging Face or in the APIs in Appendix D, while referring to their shorthands in all plots and tables for readability.

For all models, we use a temperature value of 0 to ensure a deterministic outcome.

Model Selection Process. Given the computational cost of evaluating 21 VLMs across all eight method variants (four VLM-as-planner and four VLM-as-grounder) and 150 tasks (75 per domain × 2 domains), we employ a selection procedure to identify the most capable models for our main benchmark evaluation; this ensures our benchmark is easily reproducible, and new methods only need to test their performance across the selected models.

We evaluate all 21 open-source models on two representative methods from our benchmark: the Plan variant (VLM-as-planner) and the Ground variant (VLM-as-grounder). These methods represent the core functionality of each approach without additional augmentations like CoT or memory.

For each model, we compute its average success rate across both planning methods, both ViPlan domains (ViPlan-BW and ViPlan-HH), and all three difficulty splits (simple, medium, hard). This produces a single aggregate score that reflects each model's overall capability for visual planning tasks. We then rank all models by this score and select the **top five open-source models** to form our evaluation suite for the main benchmark results.

Selection Results. Figure 4 presents the ranking of all models based on their average performance in the selection experiments. The ranking reveals clear performance tiers among VLMs, with larger models generally (though not universally)

[3]For mixture-of-experts (MoE) models, we report parameter counts using the notation {*total*}B A{*activated*}B, where *total* denotes the total number of parameters and *activated* the number of parameters used per forward pass.

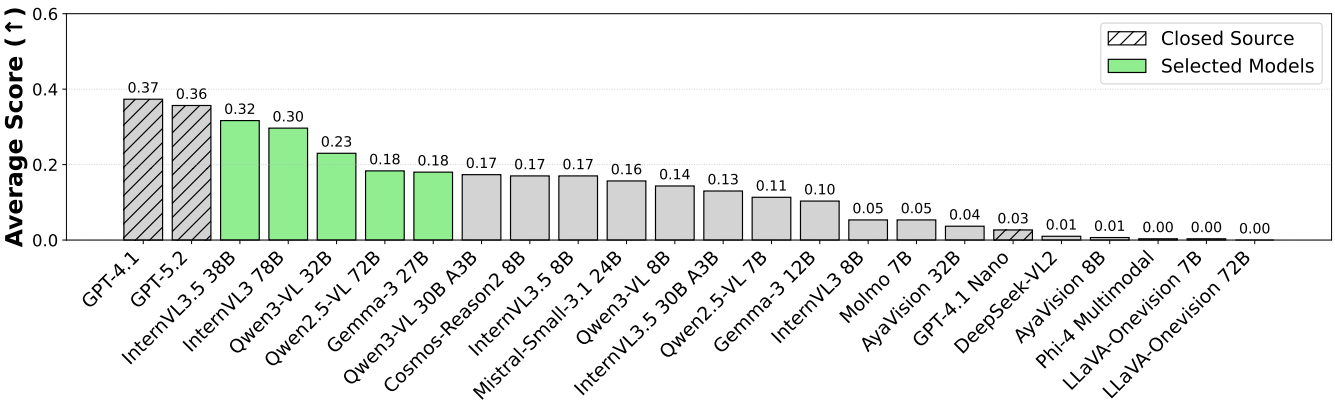


Figure 4: **Models selection.** We show a ranking of all tested VLMs, with an average score across both the Plan and Ground methods, both the ViPlan-BW and ViPlan-HH domains, and all difficulty splits. The suite of models selected for our benchmark are highlighted in green.

achieving higher scores. We also observe a small gap between large closed-source and open-source models. Interestingly, we find no significant difference between GPT-4.1 and GPT-5.2.

The five best performing models with their relative scores are: **InternVL3.5 38B** (0.32), **InternVL3 78B** (0.30), **Qwen3-VL 32B** (0.23), **Qwen2.5-VL 72B** (0.18), and **Gemma3-27B** (0.18). These VLMs, spanning different scales (27B to 78B parameters) and architectures, form the basis for all subsequent benchmark evaluations reported in Section 5.

5 Results

In this Section, we present the results of our benchmark, starting with the performances of VLM-as-planner and VLM-as-grounder variants over the two domains, then assessing the impact of CoT on the various methods and finally presenting a qualitative analysis of method failure cases. A compact overview of the results across methods and domains is available in Table 1, while Figure 5 shows a more detailed overview, separating scores across the three difficulty splits. For all methods, we report the **success rate**, i.e., the fraction of completed tasks, as a directly comparable measure of performance.

VLM-as-planner. We find VLM-as-planner performing unevenly across domains: it performs relatively well in ViPlan-HH (with the planner variants solving on average 34% of tasks), while it struggles in ViPlan-BW, with the best method (Action + CoT) surprisingly solving only 14% of tasks. When employed as planner, the VLM ideally must reason over future states, relying entirely on an implicit internal world model to simulate transitions and track state changes. However, current models often shortcut real planning and generate plausible actions by leveraging linguistic priors acquired during pretraining. These priors align more with the task structure of ViPlan-HH, while this advantage does not extend to the abstract settings of ViPlan-BW, where over-reliance on language hurts the model when the task requires precise state tracking. For both domains, performance decreases uniformly when increasing the difficulty level (see Figure 5).

VLM-as-grounder. Conversely, VLM-as-grounder has complementary strengths and weaknesses to VLM-as-planner approaches: it excels in ViPlan-BW, solving on average 46% of the tasks, yet the performance drops drastically in ViPlan-HH, reaching at most a success rate of 6%. Grounding predicates in ViPlan-BW, a simple and well-defined domain, is straightforward, as the limited visual ambiguity makes the required local yes-no questions almost trivial to answer correctly. Instead, since the ViPlan-HH PDDL domain involves a larger space of objects and predicates, verifying action preconditions and effects requires a substantially larger number of visual queries. As these are often ambiguous or confounded due to partial observability, even modest per-query errors compound over time, leading to significantly reduced overall performance. Similarly to VLM-as-planner methods, we find the performance to decrease with higher split difficulties.

Overall Comparison. Aggregated across both domains, no method or method class clearly outperforms the others, and the final gap (22% for the VLM-as-planner average against 25% for VLM-as-grounder, see Table 1) is not significant, with performance being far from saturated. Our findings rather show two complementary frameworks, with implementation details within the classes being less important. We highlight state consistency and reasoning over action outcomes as a key weakness for VLM-as-planner methods, while ambiguity and partial observability are the primary weaknesses for VLM-as-grounder methods.

Impact of Chain-of-Thought. Examining the impact of CoT more closely (Table 1), we observe that adding CoT prompting yields no consistent improvement for most methods (Action, Ground, and Ground+Mem). Surprisingly, it actively hurts performance in the Plan method for the ViPlan-HH domain, where performance drops from 39% (Plan) to 23% (Plan+CoT), while it slightly benefits the same method for the ViPlan-BW domain.

Upon manual inspection of model outputs, we find that in most failure cases, models enter repetitive thought patterns and exhaust the token budget before completing the task, despite this budget (1024 tokens) being more than ample for the planning required. This explains why the drop with Plan + CoT is more significant as the split difficulty increases (Figure 5): harder splits require longer plans and more involved reasoning,

Table 1: **Results of methods (averaged over models).** We report the average success rates (mean across the selected VLMs) for each method (as defined in Section 4.2), as well as averages across method classes; standard error of the mean across models is shown in parenthesis. The highest value in each column is bolded.

Method	ViPlan-BW	ViPlan-HH	Combined
Plan	0.07 (0.03)	0.39 (0.05)	0.23 (0.02)
Plan + CoT	0.10 (0.04)	0.23 (0.04)	0.16 (0.03)
Action	0.07 (0.03)	**0.41 (0.03)**	0.24 (0.01)
Action + CoT	0.14 (0.04)	0.35 (0.04)	0.25 (0.03)
VLM-as-planner Avg.	0.09 (0.02)	0.34 (0.02)	0.22 (0.01)
Ground	**0.47 (0.14)**	0.04 (0.01)	0.25 (0.07)
Ground + CoT	0.44 (0.14)	0.06 (0.02)	0.25 (0.07)
Ground + Mem	**0.47 (0.16)**	0.05 (0.02)	**0.26 (0.08)**
Ground + Mem + CoT	0.45 (0.14)	0.05 (0.01)	0.25 (0.07)
VLM-as-grounder Avg.	0.46 (0.07)	0.05 (0.01)	0.25 (0.04)

increasing the chance of exhausting the budget. We show an example of this behavior in Appendix A.

We conclude that when the task requires more complex reasoning (reflecting on a multi-step plan, versus selecting a single action or answering a yes-no question) or the prompt is longer (as in ViPlan-HH versus ViPlan-BW), current VLMs struggle to maintain coherent reasoning chains during CoT, which we find to be in line with recent findings (Chen *et al.*, 2024; Shiri *et al.*, 2024).

Qualitative Failure Analysis. We manually inspect several unsuccessful planning attempts to identify the most common failure modes.

For VLM-as-planner methods, agents typically fail an episode either by repeatedly predicting non-executable actions or by failing to follow the required output format (resulting in parsing errors). The first failure mode is more prevalent in ViPlan-BW than ViPlan-HH, in line with the relative performance of direct VLM planning in these domains. Instead, the second failure mode depends mainly on the model and affects the success rate equally across both domains. We find the most recent models in our evaluation suite, Qwen3-VL 32B and InternVL3.5 38B, to be particularly susceptible to this issue, echoing concerns that fine-tuning models on reasoning traces and reasoning tasks can lead to counterproductive overthinking (Cuadron *et al.*, 2025).

For VLM-as-grounder methods, failures are primarily driven by incorrect grounding of action preconditions, with asymmetric consequences depending on the direction of the error. When valid preconditions are incorrectly invalidated, the corresponding action is never executed, often leading to dead-end or terminal symbolic states. Conversely, when invalid preconditions are mistakenly validated, the agent repeatedly attempts illegal actions, resulting in persistent execution failures. This can potentially be mitigated by the use of memory in the VLM's context, as done in both Ground + Mem variants, but in practice the results do not change significantly.

In ViPlan-BW, such errors are typically due to clear model mistakes, for example misidentifying whether a block is free to move. However, in ViPlan-HH grounding errors are exacerbated by partial observability, viewpoint-dependent ambiguity

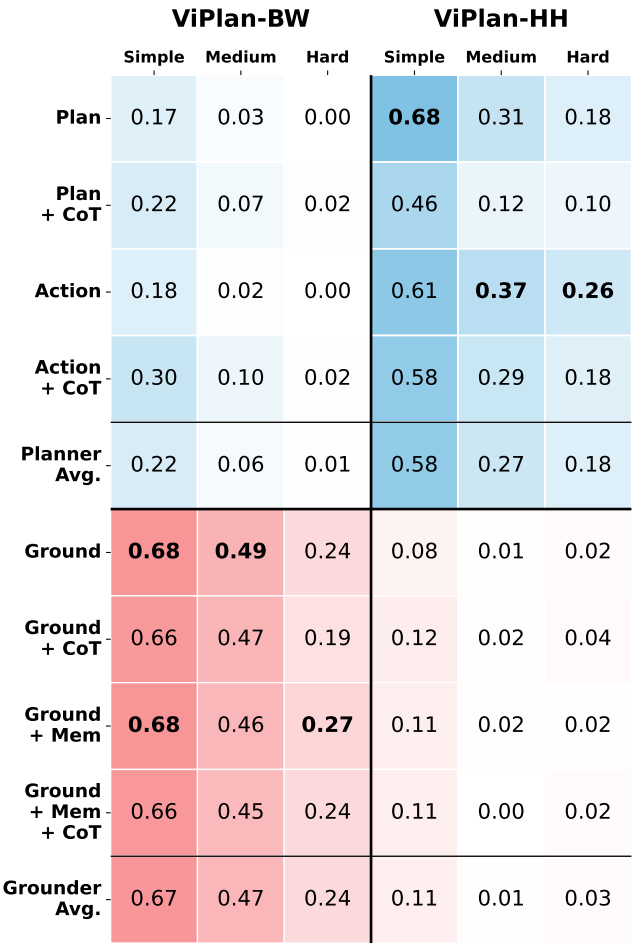


Figure 5: **Method performances across difficulty splits.** We show the average scores across models for each tested method, as well as the VLM-as-planner and VLM-as-grounder overall averages (rows), for each individual difficulty split of the domains (columns). Bolded values indicate the highest score for each column.

(e.g., determining whether an object is within reach), and by the need to verify a larger number of interacting predicates. We visualize examples of all these failure cases and expand this analysis in Appendix A.

6 Conclusions

In this work, we introduced ViPlan, the first open-source benchmark for comparing VLM-grounded symbolic approaches (VLM-as-grounder) with direct VLM planning methods (VLM-as-planner). ViPlan tests eight method variants from these two classes, across two interactive domains with complementary perceptual and reasoning demands. We find that VLM-as-planner methods are more effective when they can leverage linguistic priors, whereas VLM-as-grounder methods are better suited to tasks that require precise state grounding and explicit symbolic structure. These differences are further supported by our qualitative analysis, which sheds light on concrete failure cases that each method should tackle to improve its performance. Furthermore, we show that

Chain-of-Thought prompting provides little to no improvement across the majority of models and approaches, highlighting the ongoing difficulties with current VLMs and visual reasoning.

Overall, our benchmark presents itself as an unsolved challenge, calling for novel robust planning methods capable of both classic planning rigor and handling ambiguous situations. Finally, our work is not free of limitations, which we discuss in depth in Appendix B.

References

Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.

Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. VQA: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015.

Masataro Asai. Photo-realistic blocksworld dataset. *arXiv preprint arXiv:1812.01818*, 2018.

Ashay Athalye, Nishanth Kumar, Tom Silver, Yichao Liang, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Predicate invention from pixels via pretrained vision-language models. *arXiv preprint arXiv:2501.00296*, 2024.

Maximilian Augustin, Yannic Neuhaus, and Matthias Hein. DASH: Detection and assessment of systematic hallucinations of vlms. *arXiv preprint arXiv:2503.23573*, 2025.

Guy Azran, Yuval Goshen, Kai Yuan, and Sarah Keren. S3E: Semantic symbolic state estimation with vision-language foundation models. In *AAAI 2025 Workshop LM4Plan*, 2025.

Fahiem Bacchus and Qiang Yang. The downward refinement property. In *IJCAI*, pages 286–293, 1991.

Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge et al. Qwen3-vl technical report, 2025.

Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibo Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang et al. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*, 2025.

Rishi Bommasani, Drew A. Hudson, Ehsan Adeli, Russ Altman, Simran Arora, Sydney von Arx, Michael S. Bernstein, Jeannette Bohg, Antoine Bosselut, Emma Brunskill et al. On the opportunities and risks of foundation models. *arXiv preprint arXiv:2108.07258*, 2021.

Siwei Chen, Anxing Xiao, and David Hsu. LLM-state: Open world state representation for long-horizon task planning with large language model. *arXiv preprint arXiv:2311.17406*, 2023.

Yi Chen, Yuying Ge, Yixiao Ge, Mingyu Ding, Bohao Li, Rui Wang, Ruifeng Xu, Ying Shan, and Xihui Liu. EgoPlan-Bench: Benchmarking multimodal large language models for human-level planning. *arXiv preprint arXiv:2312.06722*, 2023.

Yangyi Chen, Karan Sikka, Michael Cogswell, Heng Ji, and Ajay Divakaran. Measuring and improving chain-of-thought reasoning in vision-language models. In *NAACL 2024*, 2024.

CohereLabs. Aya Vision: Expanding the worlds AI can see, March 2025.

Alejandro Cuadron, Dacheng Li, Wenjie Ma, Xingyao Wang, Yichuan Wang, Siyuan Zhuang, Shu Liu, Luis Gaspar Schroeder, Tian Xia, Huanzhi Mao et al. The danger of overthinking: Examining the reasoning-action dilemma in agentic tasks, 2025.

Xuzhe Dang, Lada Kudláčková, and Stefan Edelkamp. Planning with vision-language models and a use case in robot-assisted teaching. In *AAAI 2025 Workshop LM4Plan*, 2025.

Matt Deitke, Christopher Clark, Sangho Lee, Rohun Tripathi, Yue Yang, Jae Sung Park, Mohammadreza Salehi, Niklas Muennighoff, Kyle Lo, Luca Soldaini et al. Molmo and PixMo: Open weights and open data for state-of-the-art multimodal models. *arXiv preprint arXiv:2409.17146*, 2024.

Yuqing Du, Ksenia Konyushkova, Misha Denil, Akhil Raju, Jessica Landon, Felix Hill, Nando de Freitas, and Serkan Cabi. Vision-language models as success detectors. In Sarath Chandar, Razvan Pascanu, Hanie Sedghi, and Doina Precup, editors, *Proceedings of The 2nd Conference on Lifelong Learning Agents*, volume 232 of *Proceedings of Machine Learning Research*, pages 120–136. PMLR, 22–25 Aug 2023.

Jiafei Duan, Wilbert Pumacay, Nishanth Kumar, Yi Ru Wang, Shulin Tian, Wentao Yuan, Ranjay Krishna, Dieter Fox, Ajay Mandlekar, and Yijie Guo. AHA: A vision-language-model for detecting and reasoning over failures in robotic manipulation. *arXiv preprint arXiv:2410.00371*, 2024.

Malik Ghallab, Dana S. Nau, and Paolo Traverso. *Automated Planning: Theory and Practice*. Elsevier, 2004.

Lin Guan, Karthik Valmeekam, Sarath Sreedharan, and Subbarao Kambhampati. Leveraging pre-trained large language models to construct and utilize world models for model-based task planning. *Advances in Neural Information Processing Systems*, 36:79081–79094, 2023.

Yanjiang Guo, Yen-Jen Wang, Lihan Zha, and Jianyu Chen. DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment. In *IROS 2024*, 2024.

Malte Helmert. The fast downward planning system. *Journal of Artificial Intelligence Research*, 26:191–246, 2006.

Yingdong Hu, Fanqi Lin, Tong Zhang, Li Yi, and Yang Gao. Look before you leap: Unveiling the power of GPT-4V in robotic vision-language planning. In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.

Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. Language models as zero-shot planners: Extracting

actionable knowledge for embodied agents. In *International conference on machine learning*, pages 9118–9147. PMLR, 2022.

Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar et al. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning*, pages 1769–1782. PMLR, 2023.

Xu Huang, Weiwen Liu, Xiaolong Chen, Xingmei Wang, Hao Wang, Defu Lian, Yasheng Wang, Ruiming Tang, and Enhong Chen. Understanding the planning of LLM agents: A survey. *arXiv preprint arXiv:2402.02716*, 2024.

Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Saldyt, and Anil Murthy. Position: LLMs can't plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org, 2024.

Chengshu Li, Fei Xia, Roberto Martín-Martín, Michael Lingelbach, Sanjana Srivastava, Bokui Shen, Kent Elliott Vainio, Cem Gokmen, Gokul Dharan, Tanish Jain et al. igibson 2.0: Object-centric simulation for robot learning of everyday household tasks. In Aleksandra Faust, David Hsu, and Gerhard Neumann, editors, *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 455–465. PMLR, 08–11 Nov 2022.

Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.

Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 292–305, 2023.

Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Wensi Ai, Benjamin Martinez et al. Behavior-1k: A human-centered, embodied ai benchmark with 1,000 everyday activities and realistic simulation. *arXiv preprint arXiv:2403.09227*, 2024.

Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu et al. LLaVA-onevision: Easy visual task transfer. *Transactions on Machine Learning Research*, 2025.

Yichao Liang, Nishanth Kumar, Hao Tang, Adrian Weller, Joshua B Tenenbaum, Tom Silver, João F Henriques, and Kevin Ellis. Visualpredicator: Learning abstract world models with neuro-symbolic predicates for robot planning. *arXiv preprint arXiv:2410.23156*, 2024.

Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.

Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.

Hanchao Liu, Wenyuan Xue, Yifei Chen, Dapeng Chen, Xiutian Zhao, Ke Wang, Liping Hou, Rongjun Li, and Wei Peng. A survey on hallucination in large vision-language models. *arXiv preprint arXiv:2402.00253*, 2024.

Xiao Liu, Tianjie Zhang, Yu Gu, Iat Long Iong, Song XiXuan, Yifan Xu, Shudan Zhang, Hanyu Lai, Jiadai Sun, Xinyue Yang et al. VisualAgentBench: Towards large multimodal models as visual foundation agents. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

Arjun Majumdar, Anurag Ajay, Xiaohan Zhang, Pranav Putta, Sriram Yenamandra, Mikael Henaff, Sneha Silwal, Paul Mcvay, Oleksandr Maksymets, Sergio Arnaud et al. OpenEQA: Embodied question answering in the era of foundation models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16488–16498, 2024.

Drew McDermott, Malik Ghallab, Adele Howe, Craig Knoblock, Ashwin Ram, Manuela Veloso, Daniel Weld, and David Wilkins. Pddl-the planning domain definition language. 1998.

Drew M McDermott. The 1998 AI planning systems competition. *AI magazine*, 21(2):35–35, 2000.

Aoran Mei, Guo-Niu Zhu, Huaxiang Zhang, and Zhongxue Gan. ReplanVLM: Replanning robotic tasks with visual language models. *IEEE Robotics and Automation Letters*, 2024.

Andrea Micheli, Arthur Bit-Monnot, Gabriele Röger, Enrico Scala, Alessandro Valentini, Luca Framba, Alberto Rovetta, Alessandro Trapasso, Luigi Bonassi, Alfonso Emilio Gerevini et al. Unified Planning: Modeling, manipulating and solving ai planning problems in python. *SoftwareX*, 29:102012, 2025.

Microsoft, :, Abdelrahman Abouelenin, Atabak Ashfaq, Adam Atkinson, Hany Awadalla, Nguyen Bach, Jianmin Bao, Alon Benhaim, Martin Cai et al. Phi-4-mini technical report: Compact yet powerful multimodal language models via mixture-of-loras. *arXiv preprint arXiv:2503.01743*, 2025.

Toki Migimatsu and Jeannette Bohg. Grounding predicates through actions. In *2022 International Conference on Robotics and Automation (ICRA)*, pages 3498–3504. IEEE, 2022.

Mistral AI. Mistral small 3.1, March 2025.

Mitja Nikolaus, Emmanuelle Salin, Stephane Ayache, Abdellah Fourtassi, and Benoit Favre. Do vision-and-language transformers learn grounded predicate-noun dependencies? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 1538–1555, 2022.

NVIDIA. Nvidia cosmos reason 2, 2025. GitHub repository and documentation for the Cosmos Reason 2 model family.

OpenAI. Introducing GPT-4.1 in the API, April 2025.

OpenAI. Introducing GPT-5.2, December 2025.

Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.

Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley et al. Robots that ask for help: Uncertainty alignment for large language model planners. In *Conference on Robot Learning*, pages 661–682. PMLR, 2023.

Fatemeh Shiri, Xiao-Yu Guo, Mona Golestan Far, Xin Yu, Reza Haf, and Yuan-Fang Li. An empirical analysis on spatial reasoning capabilities of large multimodal models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 21440–21455, Miami, Florida, USA, November 2024. Association for Computational Linguistics.

Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M Sadler, Wei-Lun Chao, and Yu Su. LLM-planner: Few-shot grounded planning for embodied agents with large language models. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 2998–3009, 2023.

Sanjana Srivastava, Chengshu Li, Michael Lingelbach, Roberto Martín-Martín, Fei Xia, Kent Vainio, Zheng Lian, Cem Gokmen, Shyamal Buch, C. Karen Liu et al. Behavior: Benchmark for everyday household activities in virtual, interactive, and ecological environments. In *Conference on robot learning*, pages 477–490. PMLR, 2022.

Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière et al. Gemma 3 technical report. *arXiv preprint arXiv:2503.19786*, 2025.

Tristan Thrush, Ryan Jiang, Max Bartolo, Amanpreet Singh, Adina Williams, Douwe Kiela, and Candace Ross. Winoground: Probing vision and language models for visio-linguistic compositionality. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5238–5248, 2022.

Shengbang Tong, Zhuang Liu, Yuexiang Zhai, Yi Ma, Yann LeCun, and Saining Xie. Eyes wide shut? exploring the visual shortcomings of multimodal LLMs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9568–9578, 2024.

Karthik Valmeekam, Matthew Marquez, Alberto Olmo, Sarath Sreedharan, and Subbarao Kambhampati. PlanBench: An extensible benchmark for evaluating large language models on planning and reasoning about change. *Advances in Neural Information Processing Systems*, 36:38975–38987, 2023.

Karthik Valmeekam, Matthew Marquez, Sarath Sreedharan, and Subbarao Kambhampati. On the planning abilities of large language models-a critical investigation. *Advances in Neural Information Processing Systems*, 36:75993–76005, 2023.

Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao et al. Internvl3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency, 2025.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Jimmy Wu, Rika Antonova, Adam Kan, Marion Lepert, Andy Zeng, Shuran Song, Jeannette Bohg, Szymon Rusinkiewicz, and Thomas Funkhouser. Tidybot: Personalized robot assistance with large language models. *Autonomous Robots*, 47(8):1087–1102, 2023.

Qiucheng Wu, Handong Zhao, Michael Saxon, Trung Bui, William Yang Wang, Yang Zhang, and Shiyu Chang. VSP: Assessing the dual challenges of perception and reasoning in spatial planning tasks for VLMs. *arXiv preprint arXiv:2407.01863*, 2024.

Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang et al. Deepseek-VL2: Mixture-of-experts vision-language models for advanced multimodal understanding. *arXiv preprint arXiv:2412.10302*, 2024.

Zhutian Yang, Caelan Reed Garrett, Dieter Fox, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. Guiding long-horizon task and motion planning with vision language models. In *2nd CoRL Workshop on Learning Effective Abstractions for Planning*, 2024.

Rui Yang, Hanyang Chen, Junyu Zhang, Mark Zhao, Cheng Qian, Kangrui Wang, Qineng Wang, Teja Venkat Koripella, Marziyeh Movahedi, Manling Li et al. EmbodiedBench: Comprehensive benchmarking multi-modal large language models for vision-driven embodied agents. In *Proceedings of the International Conference on Machine Learning (ICML)*, 2025.

Yunan Zeng, Yan Huang, Jinjin Zhang, Zequn Jie, Zhenhua Chai, and Liang Wang. Investigating compositional challenges in vision-language models for visual grounding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14141–14151, 2024.

Xiaohan Zhang, Yan Ding, Saeid Amiri, Hao Yang, Andy Kaminski, Chad Esselink, and Shiqi Zhang. Grounding classical task planners via vision-language models. *arXiv preprint arXiv:2304.08587*, 2023.

Xiaohan Zhang, Zainab Altaweel, Yohei Hayamizu, Yan Ding, Saeid Amiri, Hao Yang, Andy Kaminski, Chad Esselink, and Shiqi Zhang. DKPROMPT: Domain knowledge

prompting vision-language models for open-world planning. In *AAAI 2025 Workshop LM4Plan*, 2025.

Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Hao Tian, Yuchen Duan, Weijie Su, Jie Shao et al. InternVL3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

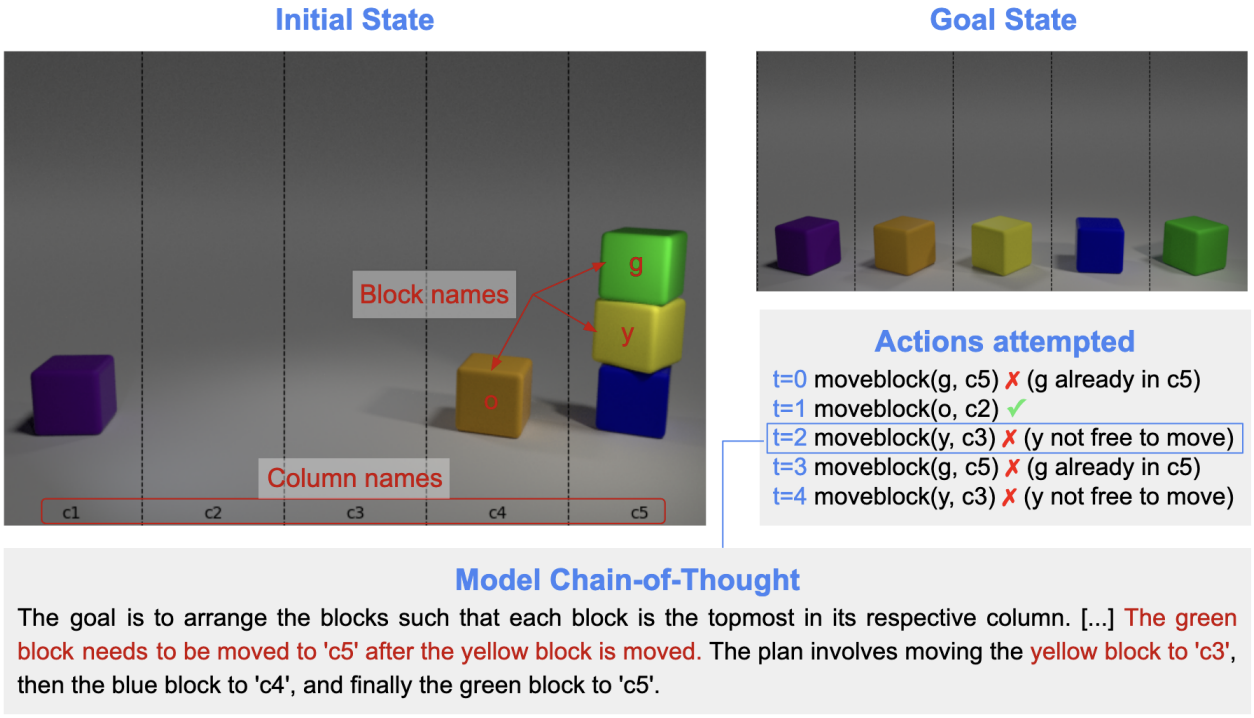


Figure 6: **Failure case of a VLM-as-planner method in ViPlan-BW.** The agent oscillates between multiple non-executable actions due to inconsistent implicit state tracking. After one successful move, the model repeatedly attempts to move blocks that are either not free to move, or already in the target column, violating action preconditions and failing to make progress toward the goal.

A Failure Cases Analysis

In this Section, we present a series of representative failure cases of both classes of methods across the two ViPlan domains that complement our analysis in the main paper. The main failure cases for VLM-as-planner and VLM-as-grounder can happen regardless of whether CoT is employed. We report cases accompanied by CoTs (edited for compactness and clarity), as we find them insightful. We additionally present one failure case specific of VLM-as-planner methods that employ CoT.

A.1 VLM-as-planner.

VLM-as-planner variants failure cases can be divided in actions that are parsable but non-executable and actions that fail to be parsed, which we present in this order.

Non-executable action failures. We observe two main dynamics that arise from a single failed action and propagate into a sequence of failures until the termination of an episode:

1. The agent oscillates between two non-executable actions, as shown in Figure 6.
2. The same action is repeated again and again, as possible to see in Figure 7.

In both cases, a mistaken diagnosis about why the action is non-executable is responsible for these failing dynamics. It is possible that moving to more advanced VLM agents with better memory and replanning procedures could solve some of these failures.

Non-parsable action failures. As we require all model outputs to be in JSON format, all parsing errors are due to incorrect formatting and we are not aware of any shortcomings of our parsing procedure that would discard otherwise valid outputs. Ultimately, outputs with the wrong format can be re-conducted to the inability of a model to follow instructions. We inspect one particularly interesting case of parsing error (see Figure 8), caused by the model CoT repeating itself in loops until it exhausts the token budget without producing any valid JSON output. This case is observed especially in the Plan+CoT variant of VLM-as-planner, as it tends to arise from the interaction between the request for a plan in the prompt and the usage of CoT.

A.2 VLM-as-grounder.

In VLM-as-grounder, there can be errors in assessing the preconditions or the effects of an action. We observe that episodes become irrecoverable for symbolic planners when preconditions are misjudged, entering one of two states:

1. A precondition is erroneously judged as unsatisfied, thus preventing a valid action that is necessary to achieve the goal (see Figure 9). The precondition then remains unsatisfied also at future steps, leading to a dead-end state.

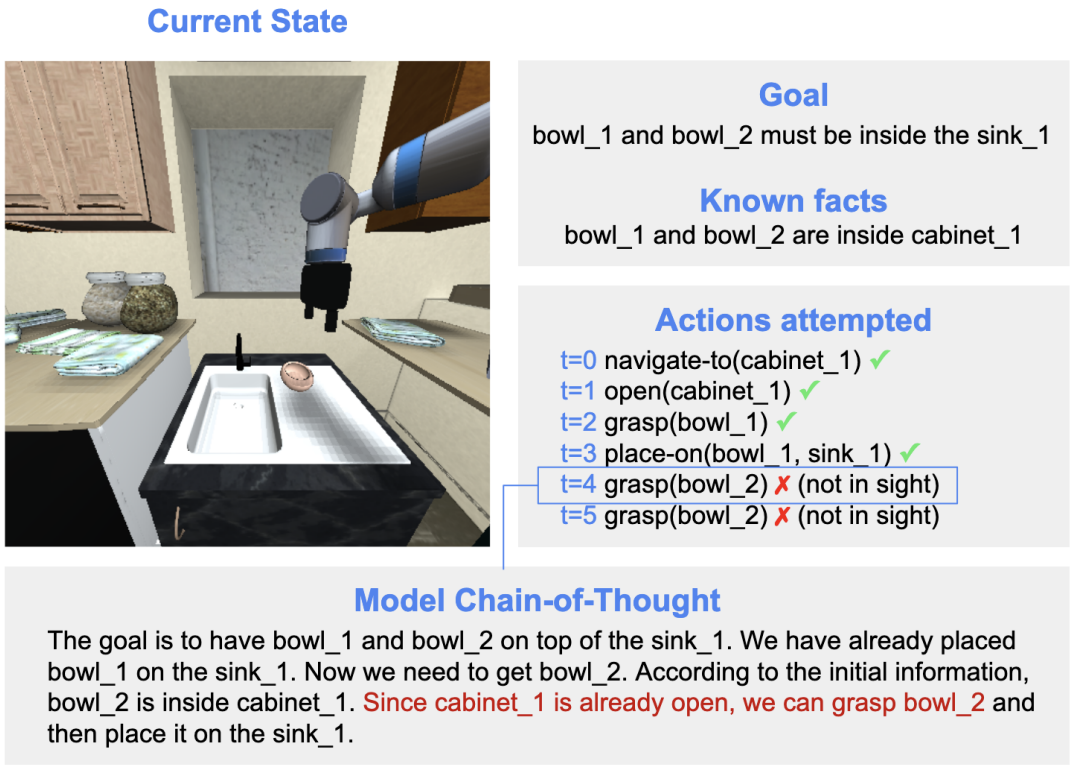


Figure 7: **Failure case of a VLM-as-planner method in ViPlan-HH.** The agent attempts a non-executable action. After the first attempt, it fails to diagnose the cause for the failure and enters a loop of re-attempts.

2. A precondition is erroneously judged as satisfied, thus attempting an invalid action (see Figure 10). The precondition then remains satisfied also at future steps, leading to more attempts and potential infinite loops.

Notice that an action can only be chosen if the estimated symbolic state satisfies the preconditions. Thus to enter the second case, some predicate must be wrong.

Errors in checking the effects of actions manifest in wrong assumptions about preconditions in the future steps and can thus lead to the second case above.

B Limitations

In terms of method classes, we assume that in ViPlan-HH some privileged information is given to the VLM both when serving as a planner and as a grounder. This is because a scenario where the model has no information about where certain objects can be found would require a big amount of exploration. It is outside the scope of this work to determine how to avoid using privileged information, but future directions should investigate how to lift this assumption, especially in the case of VLM-as-grounder, where PDDL is not easily extendable to unobserved predicates otherwise.

We assume a hierarchical control interface where low-level controllers handle action execution and termination. Consequently, if an agent attempts an action whose preconditions are not met in the ground truth state, the execution is rejected, the state remains unchanged, and the agent receives feedback to trigger replanning; this aligns with standard TAMP architectures where infeasible refinements cause backtracking. Furthermore, we assume the low-level controller signals when an action has terminated, tasking the VLM solely with verifying the high-level effects or selecting the next step, although recent work suggests VLMs could also assist in detecting termination conditions (Du *et al.*, 2023).

More in general, ViPlan-HH presents non-negligible stochasticity which could lead to fluctuations in the results, although these are averaged out by the 25 problems per split that we consider. Due to our computational budget and the large amount of models tested (see Appendix G for a breakdown of all the computational resources needed for the experiments), we could not run multiple seeds for each experiment, but this would be advisable in future experiments with the benchmark.

We also assume, as is typically the case with classical planning literature, that the domain $\mathcal{D}$ is given. However recent works, such as Guan *et al.* (2023), explore how the domain could be generated using LLMs. We also relied on templates to translate grounded predicates into questions. Recent work (Azran *et al.*, 2025) has shown how this can be automated with LLMs, but as the scope of our benchmark is evaluating VLMs, we avoided this for simplicity.

Chain-of-Thought output (excerpt)

[Initial reasoning prefix]
The current state shows that the purple block is already on top of the red block in column c3, which satisfies part of the goal. The orange block is currently in column c3, but it needs to be moved to column c5. The green block is in column c4, which is correct, but it needs to be the topmost block in its column.

[Recurring reasoning block]
The blue block is in column c1, but it needs to be in column c2 and be the topmost block there. The purple block is already in column c3, which is correct, but it needs to be the topmost block in its column. The red block is already in column c3, which is correct, but it needs to be under the purple block. The yellow block was incorrectly moved to column c5, but it should not be there. The green block is already in column c4, which is correct, but it needs to be the topmost block in its column.

[Recurring reasoning block - identical to above]
The blue block is in column c1, but it needs to be in column c2 and be the topmost block there. The purple block is already in column c3, which is correct, but it needs to be the topmost block in its column. The red block is already in column c3, which is correct, but it needs to be under the purple block. The yellow block was incorrectly moved to column c5, but it should not be there. The green block is already in column c4, which is correct, but it needs to be the topmost block in its column.

[The above reasoning block is repeated verbatim 24 times, without producing any new action, until the token budget is exhausted.]

Figure 8: **Failure case illustrating the negative impact of Chain-of-Thought prompting in ViPlan-BW.** The VLM initially produces a coherent reasoning prefix, but its reasoning then converges to a recurring reasoning block that is repeated verbatim across subsequent steps, preventing the generation of new actions and exhausting the token budget.

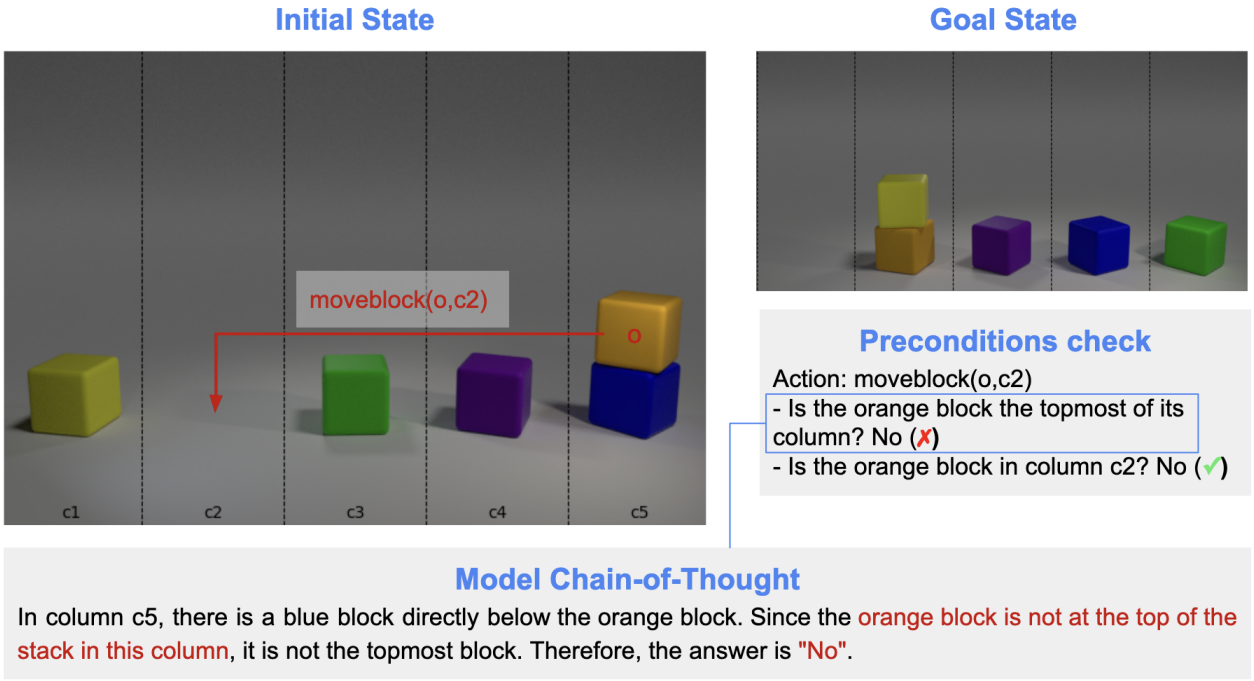


Figure 9: **Failure case of a VLM-as-grounder method in ViPlan-BW.** During precondition grounding, the VLM incorrectly judges a required precondition of the action `moveblock(o, c2)` as unsatisfied, stating that the orange block is not topmost in its column when it actually is. As a result, the symbolic planner rejects an otherwise valid action, leading to a dead-end state in which no valid plan can be generated and the episode terminates unsuccessfully.

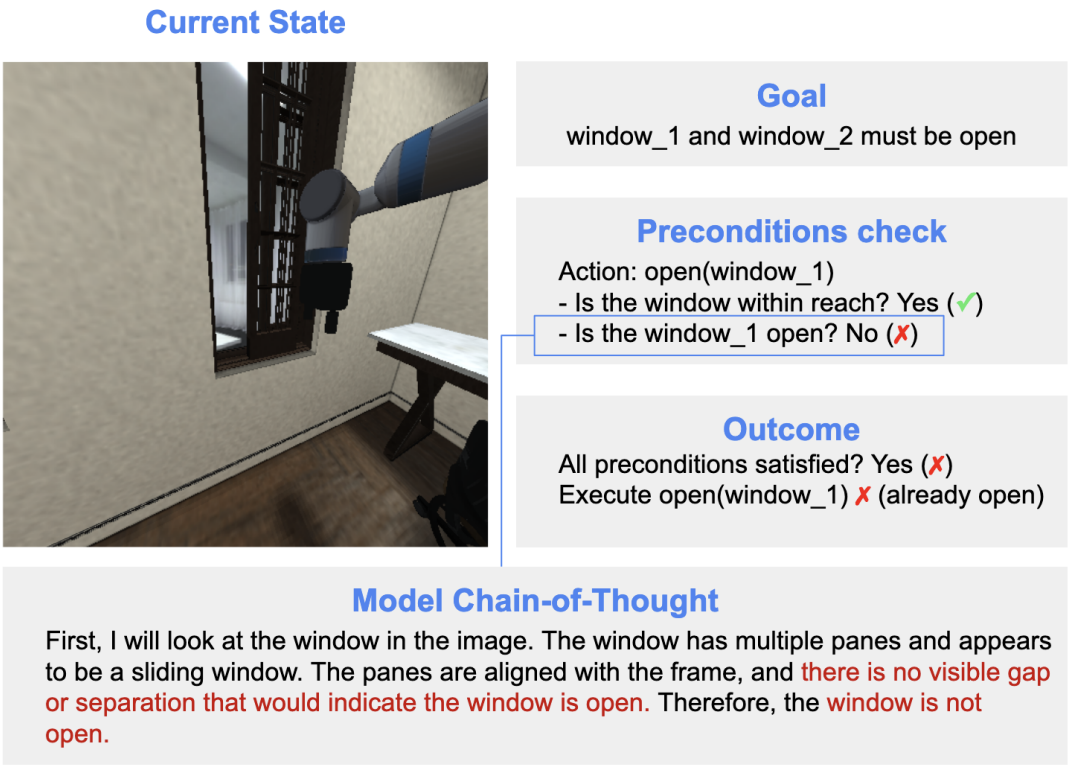


Figure 10: **Failure case of a VLM-as-grounder method in ViPlan-HH.** During precondition grounding, the VLM incorrectly judges a required precondition of the action `open(window_1)` as satisfied, stating that the window is closed when it is already open. Consequently, the symbolic planner attempts an invalid action that cannot be executed in the environment. If uncorrected, this error can lead to repeated invalid action attempts and potential infinite loops.

C Additional Related Work

The following is an expansion on Related Work (Section 3 of the main paper) with literature referring to Benchmarks for Vision-Language Models that evaluate their planning abilities and robustness to hallucinations.

Benchmarks for Vision-Language Models. Our work closely aligns with other benchmarks, specifically in the area of planning, object hallucinations and relationship hallucinations with VLMs. For planning, previous research typically framed the problem as Visual Question Answering (VQA) (Antol *et al.*, 2015), asking questions about which actions to take (Chen *et al.*, 2023b; Majumdar *et al.*, 2024). Visual Spatial Planning (VSP) (Wu *et al.*, 2024a) specifically focuses on spatial relationships, but without any notion of symbolic predicates. Our work is closer to PlanBench (Valmeekam *et al.*, 2023a), which investigates the combination of LLMs with symbolic planning, but without considering the visual component.

Hallucinations in foundation models are a widely studied phenomenon (Liu *et al.*, 2024). POPE (Li *et al.*, 2023b) is an established benchmark for object hallucination in VLMs, with adversarial examples constructed out of frequently co-occurring objects. This kind of evaluation was scaled up by DASH (Augustin *et al.*, 2025) through smart retrieval from a bigger dataset. MMVP (Tong *et al.*, 2024) investigates both object and relationship hallucinations by looking at *CLIP-blind* pairs of images, placing a large emphasis on the role of the vision encoder (Radford *et al.*, 2021). Benchmarks for relationship hallucination typically work in a similar way, by presenting pairs of images with adversarial captions (Thrush *et al.*, 2022; Nikolaus *et al.*, 2022), with later advancements also focusing on the particular role of the vision encoder (Zeng *et al.*, 2024).

D Model Information

For clarity and readability, we report simplified model names throughout the main text and figures. In Table 2 are the correspondance between the full names of the models and the simplified versions. For our experiments, GPT-5.2, GPT-4.1 and GPT-4.1 Nano were accessed via the OpenAI API[1], while all other models were downloaded from the Hugging Face Hub[2].

Table 2: **Simplified Model Names.** Mapping between specific model identifiers and simplified model names used in the paper.

Identifier	Simplified Name	Reference
`CohereLabs/aya-vision-8b`	AyaVision 8B	(CohereLabs, 2025)
`CohereLabs/aya-vision-32b`	AyaVision 32B	(CohereLabs, 2025)
`nvidia/Cosmos-Reason2-8B`	Cosmos-Reason2 8B	(NVIDIA, 2025)
`deepseek-ai/deepseek-vl2`	DeepSeek-VL2	(Wu *et al.*, 2024b)
`gpt-5.2-2025-12-11`	GPT-5.2	(OpenAI, 2025b)
`gpt-4.1-nano-2025-04-14`	GPT-4.1 Nano	(OpenAI, 2025a)
`gpt-4.1-2025-04-14`	GPT-4.1	(OpenAI, 2025a)
`google/gemma-3-12b-it`	Gemma-3 12B	(Team *et al.*, 2025)
`google/gemma-3-27b-it`	Gemma-3 27B	(Team *et al.*, 2025)
`OpenGVLab/InternVL3-8B`	InternVL3 8B	(Zhu *et al.*, 2025)
`OpenGVLab/InternVL3-78B`	InternVL3 78B	(Zhu *et al.*, 2025)
`OpenGVLab/InternVL3_5-8B-HF`	InternVL3.5 8B	(Wang *et al.*, 2025)
`OpenGVLab/InternVL3_5-38B-HF`	InternVL3.5 38B	(Wang *et al.*, 2025)
`OpenGVLab/InternVL3_5-30B-A3B-HF`	InternVL3.5 30B A3B	(Wang *et al.*, 2025)
`lmms-lab/llava-onevision-qwen2-7b-ov-hf`	LLaVA-Onevision 7B	(Li *et al.*, 2025)
`lmms-lab/llava-onevision-qwen2-72b-ov-hf`	LLaVA-Onevision 72B	(Li *et al.*, 2025)
`mistralai/Mistral-Small-3.1-24B-Instruct-2503`	Mistral-Small-3.1 24B	(Mistral AI, 2025)
`allenai/Molmo-7B-D-0924`	Molmo 7B	(Deitke *et al.*, 2024)
`microsoft/Phi-4-multimodal-instruct`	Phi-4 Multimodal	(Microsoft *et al.*, 2025)
`Qwen/Qwen2.5-VL-7B-Instruct`	Qwen2.5-VL 7B	(Bai *et al.*, 2025b)
`Qwen/Qwen2.5-VL-72B-Instruct`	Qwen2.5-VL 72B	(Bai *et al.*, 2025b)
`Qwen/Qwen3-VL-8B-Instruct`	Qwen3-VL 8B	(Bai *et al.*, 2025a)
`Qwen/Qwen3-VL-32B-Instruct`	Qwen3-VL 32B	(Bai *et al.*, 2025a)
`Qwen/Qwen3-VL-30B-A3B-Instruct`	Qwen3-VL 30B A3B	(Bai *et al.*, 2025a)

E Error Computations

In this Section we outline how the errors are computed throughout the paper.

[1] https://openai.com/index/openai-api/

[2] https://huggingface.co/docs/hub/index

To estimate the error of the mean on the success rate on a split, we assume each problem being a Bernoulli trial (i.e., drawn from a binomial distribution) with true success probability p. Our empirical estimate of p, denoted as $\hat{p}$, is used to estimate the standard error of the mean (SEM) as

$$\text{SEM}(\hat{p}) \approx \sqrt{(\hat{p}(1-\hat{p})/n}\,, \tag{1}$$

where n is the number of problems in the split. This normal approximation to the binomial distribution is reasonable for $n = 25$, which holds for all splits in our experiments.

In the Tables we further report the error of the average success rates across difficulty splits and occasionally even combined across the two domains of ViPlan. These are computed using standard error propagation for the average of independent estimates:

$$\text{SEM}(\bar{x}) = \text{SEM}\left(\sum_{i=1}^{m} x_i/m\right) = \frac{1}{m}\sqrt{\sum_i \text{SEM}(x_i)^2}\,. \tag{2}$$

F Statistical Significance of Chain-of-Thought

We report in Table 3 the numerical differences between the average success rate with and without CoT prompting, together with their errors. Except for Plan in the medium split of both domains, there is no significant difference (measured in 3 standard deviations of the mean from 0) in employing CoT or not, and even in this case, the difference is positive in ViPlan-BW, but negative in ViPlan-HH. We conclude that there is no evidence that CoT helps in our benchmark when averaging across models.

Table 3: **Statistical significance of CoT difference.** We report the average difference between experiments with and without CoT prompting across models for each split and the Ground and Plan methods, using all models tested during pre-evaluation. We further report the standard error of the mean, and the ratio between the absolute value of the average and its error in parenthesis. We bold ratios that are below 3, indicating no statistical difference between the two approaches.

Method	Domain	Simple		Medium		Hard	
		Average	Error	Average	Error	Average	Error
Ground	ViPLan-BW	-0.060	0.022 (**2.8**)	-0.018	0.015 (**1.2**)	-0.005	0.013 (**0.4**)
	ViPLan-HH	0.010	0.016 (**0.6**)	0.000	0.005 (**0.0**)	-0.003	0.005 (**0.5**)
Plan	ViPLan-BW	0.052	0.018 (**3.0**)	0.033	0.009 (3.6)	0.007	0.005 (**1.4**)
	ViPLan-HH	-0.068	0.024 (**2.8**)	-0.065	0.018 (3.5)	-0.040	0.017 (**2.4**)

G Computational Resources

We ran all our experiments on a cluster equipped with three different GPU models from NVIDIA: A100 with 80GB VRAM, H100 with 80GB VRAM and H200 with 141GB VRAM.

The computational resources needed to run the full benchmark for 12 models fitting in a single 80GB GPU and 9 models fitting in two GPUs, totaling around 1800 GPU hours. While this is a considerable amount, for practitioners willing to benchmark a VLM of around 7-8B parameters, this would require roughly 39 hours for just the Ground and Plan methods (with and without CoT) and less than 15 if running only the experiments without Chain-of-Thought. Benchmarking a new method on the five selected models (assuming the GPU cost is similar to one of our CoT implementations) would instead cost around 100 GPU hours (assuming all selected models are ran on a double GPU setup, but this can be reduced if, for example, they are loaded on a single H200).

More in general, we estimate that all the experiments of the project required us around 4000 GPU hours, several of which employing smaller V100 GPUs to run the iGibson simulator.

Finally, the experiments with GPT-5.2, GPT-4.1 and GPT-4.1 Nano costed around 210$.

H Domain Details

H.1 ViPlan-BW

We divided the tasks into three splits, simple, medium and hard, each composed of 25 different problems which were automatically generated and validated. The details for each split can be found in Table 4.

Table 4: **ViPlan-BW Task Splits.** Task splits based on difficulty, number of blocks, columns, and plan length.

Split	# Blocks	# Columns	Plan Length
Simple	3	4	3–5
Medium	5	5	5–10
Hard	6	4	8–15

While typical PDDL domains for Blocksworld also include a predicate for the agent holding a block in its hand, since our image observation space does not show the agent, we instead define a simplified version of the domain with only a single action that moves a block to the top of a specified column.

The full PDDL domain for ViPlan-BW can be seen in Figure 11. Note that the `rightOf` and `leftOf` predicates are never actually used in the `moveBlock` action; however they are still filled in by the VLM when enumerating all predicates, and can thus serve as a way of measuring the VLM's awareness of spatial directions such as left and right. The domain and problem files can be given to a classical planner to compute a plan.

ViPlan-BW Domain

```
(define (domain Blocksworld)
  (:requirements :strips :typing :negative-preconditions :conditional-effects
    :equality)
  (:types block column)

  (:predicates
    (on ?b1 - block ?b2 - block) ;; block b1 is on block b2
    (inColumn ?b - block ?c - column) ;; block b is in column c
    (clear ?b - block) ;; block b is clear (i.e., nothing is on top of it)
    (rightOf ?c1 - column ?c2 - column) ;; column c1 is to the right of column c2
    (leftOf ?c1 - column ?c2 - column) ;; column c1 is to the left of column c2
  )

  (:action moveBlock
    :parameters (?b1 - block ?c1 - column) ;; move block b1 to column c1
    :precondition (and (clear ?b1) (not (inColumn ?b1 ?c1))) ;; block b1 must be clear and
     not already in column c1
    :effect (and
      (forall
        (?b2 - block) ;; for all blocks b2
        (and
          (when
            (on ?b1 ?b2)
            (and (not (on ?b1 ?b2)) (clear ?b2)) ;; if block b1 was on block b2, then b1
    is no longer on b2 and b2 is clear
          )
          (when
            (and (inColumn ?b2 ?c1) (clear ?b2) (not (= ?b2 ?b1)))
            (and (on ?b1 ?b2) (not (clear ?b2))) ;; if another block b2 was in the column
    c1 where b1 is moving and b2 was clear, then b1 is now on b2 and b2 is no longer clear
          )
        )
      )
      (forall
        (?c2 - column) ;; for all columns c2
        (when
          (inColumn ?b1 ?c2)
          (not (inColumn ?b1 ?c2))) ;; if block b1 was in column c2, then b1 is no longer
    in c2)
      (inColumn ?b1 ?c1) ;; block b1 is now in column c1
      (clear ?b1) ;; block b1 is now clear (as it must be if it was moved)
    )
  )
)
```

Figure 11: **ViPlan-BW Domain.** PDDL domain for ViPlan-BW.

Example Problem for ViPlan-BW

```
(define (problem simple_problem_0)
  (:domain Blocksworld)

  (:objects
    Y P R - block
    C1 C2 C3 C4 - column
  )

  (:init


    (clear Y)
    (clear P)
    (clear R)

    (inColumn Y C2)
    (inColumn P C1)
    (inColumn R C4)

    (rightOf C2 C1)
    (rightOf C3 C2)
    (rightOf C4 C3)

    (leftOf C1 C2)
    (leftOf C2 C3)
    (leftOf C3 C4)
  )
  (:goal
    (and

      (clear Y)
      (clear P)
      (clear R)

      (inColumn Y C3)
      (inColumn P C4)
      (inColumn R C1)
    )
  )
)
```

Figure 12: **ViPlan-BW Problem.** Example of a problem for the ViPlan-BW domain.

Table 5: **Task splits used in the ViPlan-HH domain of our benchmark.** Tasks appearing in multiple splits have been adjusted in terms of goal to achieve, to make them easier or harder, depending on the split. The two additional columns report the number of problem instances per task and the minimum number of actions required to complete each instance.

Difficulty	Task	# Instances	# Actions
Simple	cleaning out drawers	5	5
	locking every door	5	4
	locking every window	5	6
	packing food for work	5	5
	sorting books	5	4
Medium	cleaning out drawers	2	10
	collect misplaced items	4	8
	packing food for work	4	10
	putting away toys	5	8
	sorting books	4	8
	sorting groceries	6	10
Hard	cleaning out drawers	5	15
	organizing boxes in garage	5	11
	organizing file cabinet	4	14
	putting away toys	4	12
	sorting groceries	7	13

H.2 ViPlan-HH

ViPlan-HH is composed of a selection of Behavior Domain Definition Language (BDDL)[3] problems from the BEHAVIOR-100 task suite (Srivastava *et al.*, 2022). As BDDL problems are not enough for planning, but are used as constraints to generate the problem layout in the iGibson simulator, we translated them into PDDL and wrote an adequate domain, reported in Figure 13.

The iGibson simulator includes multiple scenes that are compatible with each problem, and each scene has multiple instances (e.g., different dispositions of objects or different looking 3D models for the same object). Thus, in order to reach 25 problems, we select 5 tasks for the simple split, 6 tasks for the medium split and 5 tasks for the hard split. For each tasks, we write a PDDL problem file (see e.g., Figure 14) and then select multiple scenes and instances, to reach a total of 25 unique (PDDL problem, scene, scene instance) triplets, as reported in Table 5. For each split, the problems are edited (e.g., by considering more or less objects) to ensure that length of the plan required to solve them matches the one used for ViPlan-BW (Table 4). Sample images from iGibson can be found in Figures 1 and 2 of the main text.

Note that a successor to iGibson, named OmniGibson (Li *et al.*, 2024), was recently proposed. While it provides more realistic graphics, which would be an advantage for VLM evaluation, it requires a NVIDIA RTX GPU (NVIDIA RTX 2070+), causing significant accessibility issues. We thus opted for iGibson instead for ease of reuse.

ViPlan-HH Original Contributions

While we build on top of iGibson and leverage BDDL problems from BEHAVIOR-100, adapting the environment to our benchmark required months of work. Our main contributions in this regard can be summarized as follows:

1. Implement a **PDDL domain** and interface it with a symbolic planner
2. Implement in iGibson each **predicate** and **high-level action** listed in the domain by leveraging previleged low-level access to the internal states of the simulator
3. Add **3D bounding boxes with object labels** when multiple objects of the same type are in sight
4. Building a **server-client framework** to enable containerization of the iGibson environment with GPU acceleration on a SLURM server

[3]This is a simplified version of PDDL, where no domain is provided, only the problem.

ViPlan-HH Domain

```
(define (domain igibson)
    (:requirements :strips :typing :negative-preconditions :conditional-effects :equality)

    (:types
        container movable - object
    )

    (:predicates
        ;; Agent predicates
        (reachable ?o - object)
        (holding ?m - movable)

        ;; Object attributes
        (open ?c - container)

        ;; Object relations
        (ontop ?o1 - object ?o2 - object)
        (nextto ?o1 - object ?o2 - object)

        ;; Only containers can contain objects
        (inside ?o - object ?c - container)
    )

    (:action grasp
        :parameters (?m - movable)
        :precondition (and
            (reachable ?m)
            ;; Agent must not be holding anything
            (forall (?x - movable)
                (not (holding ?x))
            )
        )
        :effect (and
            (holding ?m)
            (forall (?y - object)
                (and
                    ;; If grasped object is on top of something,
                    ;; it is no longer on top of it
                    (not (ontop ?m ?y))

                    ;; Same for nextto
                    (not (nextto ?m ?y)))
            )

            ;; If m was in a container, it's not anymore
            (forall (?c - container)
                (when (inside ?m ?c) (not (inside ?m ?c)))
            )
        )
    )

    (:action place-on
        :parameters (?m - movable ?o2 - object)
        :precondition (and
            (holding ?m)
            (reachable ?o2)
        )
        :effect (and
            (ontop ?m ?o2)
            (not (holding ?m))
        )
    )
```

```
    (:action place-next-to
        :parameters (?m - movable ?o2 - object)
        :precondition (and
            (holding ?m)
            (reachable ?o2)
        )
        :effect (and
            (nextto ?m ?o2)
            (not (holding ?m))
        )
    )

    (:action place-inside
        :parameters (?m - movable ?c - container)
        :precondition (and
            (holding ?m)
            (reachable ?c)
            (open ?c)
        )
        :effect (and
            (inside ?m ?c)
            (not (holding ?m))
        )
    )

    (:action open-container
        :parameters (?c - container)
        :precondition (and
            (reachable ?c)
            (not (open ?c))
            ;; Agent must not be holding anything
            (forall (?x - movable)
                (not (holding ?x))
            )
        )
        :effect (and
            (open ?c)
            ;; All objects inside the container are reachable
            (forall (?o - object)
                (when (inside ?o ?c) (reachable ?o))
            )
        )
    )

    (:action close-container
        :parameters (?c - container)
        :precondition (and
            (reachable ?c)
            (open ?c)
        )
        :effect (and
            (not (open ?c))
            ;; All objects inside the container are unreachable
            (forall (?o - object)
                (when (inside ?o ?c) (not (reachable ?o)))
            )
        )
    )

    (:action navigate-to
        :parameters (?o - object)
        :precondition (and
```

```
            (not (reachable ?o))
            ;; Do not navigate-to things hidden in a closed container
            (forall (?c - container)
                (or (not(inside ?o ?c)) (open ?c))
            )
        )
        :effect (and
            (reachable ?o) ;; make target object reachable

            ;; Make every other object unreachable
            (forall (?x - object)
                (when (not (= ?x ?o))
                    (not (reachable ?x))))

            ;; Also, if there exists a container which is ?o and that it's open,
            ;; set the objects inside as reachable
            (forall (?c - container ?x - object)
                (when (and (= ?c ?o) (open ?c) (inside ?x ?c))
                    (reachable ?x)))
        )
    )
)
```

Figure 13: PDDL domain for the ViPlan-HH environment.

Example Problem for ViPlan-HH

```
(define (problem cleaning_out_drawers_0)
    (:domain igibson)

    (:objects
        bowl_1 - movable
        cabinet_1 - container
        sink_1 - object
    )

    (:init
        (inside bowl_1 cabinet_1)
        (not (open cabinet_1))
    )

    (:goal
        (and
            (ontop bowl_1 sink_1)
        )
    )
)
```

Figure 14: **ViPlan-HH Problem.** Example of a problem for the ViPlan-HH domain.

I Prompts

In the following, we report prompts for the Ground, Ground + CoT, Plan, and Plan + CoT methods used in our experiments. The prompts for Action and Action + CoT use the same format as Plan and Plan + CoT respectively, with the JSON only containing a single "action" element rather than a full plan (and the wording in the system prompt adjusted accordingly to ask for a single action rather than a plan). For the memory augmented Ground variants, the prompts are the same as the ones reported, with the memory context appended at the end of the user prompt. All possible memory augmentations are reported after the prompts.

Ground prompt for ViPlan-BW

<system>
You are tasked with replying to a question about the given image. You will be given a single question, defined after the keyword " Question:" and will need to answer it ONLY with Yes or No. Do not write anything else besides your answer.

The image will be about colored blocks and how they relate to each other. In the environment, the blocks will be arranged in columns, spanning from left to right. Keep in mind that some of these columns can be empty with no blocks currently placed in them. Within a column one or multiple blocks of different colors can be stacked on top of each other. Your task is to correctly evaluate the question based on the image provided.
</system>
<user>
{image}
</user>

Ground + CoT prompt for ViPlan-BW

<system>
You are tasked with replying to a question about the given image. You will be given a single question, defined after the keyword " Question:" and will need to first reason about it and then give a Yes or No answer. The reasoning for your answer should be written within the XML−style <explanation></explanation>tags. To write the final answer, you should write only "Yes" or "No" surrounded by <answer></answer>tags. Do not write anything else besides your step−by−step reasoning and your answer.

Example output for a question about the an image:
‘‘‘
Question: Is there a dog on top of a table?
<explanation>
First, I will look for a dog in the image. Then, I will check if the dog is on top of a table. In the image, there is a dog and there is a table, but the dog is not on top of the table. Therefore, the answer is "No".
</explanation>
<answer>
No
</answer>
‘‘‘
</system>
<user>
The image will be about colored blocks and how they relate to each other. In the environment, the blocks will be arranged in columns, spanning from left to right. Keep in mind that some of these columns can be empty with no blocks currently placed in them. Within a column one or multiple blocks of different colors can be stacked on top of each other.

{image}
</user>

Ground prompt for ViPlan-HH

<system>
You are tasked with replying to a question about the given image. You will be given a single question, defined after the keyword " Question:" and will need to answer it ONLY with Yes or No. Do not write anything else besides your answer.
</system>
<user>
The environment is a virtual household simulator, with objects and furniture which can be interacted with. There is a robotic arm, which is the agent, that can hold objects.
{image}
</user>

Ground + CoT prompt for ViPlan-HH

<system>

You are tasked with replying to a question about the given image. You will be given a single question, defined after the keyword " Question:" and will need to first reason about it and then give a Yes or No answer. The reasoning for your answer should be written within the XML−style <explanation></explanation>tags. To write the final answer, you should write only "Yes" or "No" surrounded by <answer></answer>tags. Do not write anything else besides your step−by−step reasoning and your answer.

Example output for a question about the an image:
‘‘‘
Question: Is there a dog on top of a table?
<explanation>
First, I will look for a dog in the image. Then, I will check if the dog is on top of a table. In the image, there is a dog and there is a table, but the dog is not on top of the table. Therefore, the answer is "No".
</explanation>
<answer>
No
</answer>
‘‘‘
</system>

<user>
The environment is a virtual household simulator, with objects and furniture which can be interacted with. There is a robotic arm, which is the agent, that can hold objects.

{image}
</user>

Plan prompt for ViPlan-BW

<system>
You are an expert planning assistant. You will be given an image which represents the current state of the environment you are in, a natural language description of the goal that needs to be achieved and a set of actions that can be performed in the environment.
Your task is to generate a plan that achieves the goal, in the form of a sequence of actions that need to be executed to reach the goal.
The format of your output should be a JSON object with the following structure:
‘‘‘json
```
{
   "plan": [
      {
            "action": action_name,
            "parameters": {
                  parameter_name: parameter_value
            }
      },
      ... other actions ...
      ]
}
```
‘‘‘

You will also receive feedback of the previously taken actions, with a note showing if they failed or not. If an action failed, think about why that could be and then output a new plan accordingly.
</system>
<user>
Description of the environment
The environment is about colored blocks and how they relate to each other. In the environment, the blocks will be arranged in columns, spanning from left to right. Keep in mind that some of these columns can be empty with no blocks currently placed in them. Within a column one or multiple blocks of different colors can be stacked on top of each other. Your task is to correctly evaluate the question based on the image provided.

Available actions
You have only one action available, called ‘moveblock(block, column)‘. This action allows you to move a block from its current column to the specified column. In order to perform this action, the block you want to move must be the topmost block of its column and must not already be in the target column. If the action is valid, the block will be moved to the specified column and will be placed on top of any blocks that are already in that column, if any.

To refer to the blocks, use lowercase letters for the colors: 'r' for red, 'g' for green, 'b' for blue, 'y' for yellow, 'p' for purple, 'o' for orange. To refer to the columns, use the labels provided in the image, 'c1', 'c2', 'c3', 'c4' and 'c5'.

Goal
{goal_string}

Previously taken actions
{previous_actions}

Current environment state
{image}
</user>

Plan + CoT prompt for ViPlan-BW

<system>
You are an expert planning assistant. You will be given an image which represents the current state of the environment you are in, a natural language description of the goal that needs to be achieved and a set of actions that can be performed in the environment. Your task is to generate a plan that achieves the goal, in the form of a sequence of actions that need to be executed to reach the goal. Before answering with the plan, think carefully step by step about the actions you need to take and what the expected outcome of each action is. Write the reasoning behind the plan and justify each action you are going to take. Make sure that each action is possible, and if previous actions failed, reason about why this could be the case.

The format of your output should be a JSON object with the following structure. Make sure that the explanation is also written inside the json.
```json
{
  "explanation": <a detailed explanation of the plan>,
  "plan": [
    {
        "action": action_name,
        "parameters": {
            parameter_name: parameter_value
        }
    },
    ... other actions ...
    ]
}
```

You will also receive feedback of the previously taken actions, with a note showing if they failed or not. If an action failed, think about why that could be and then output a new plan accordingly.
</system>
<user>
Description of the environment
The environment is about colored blocks and how they relate to each other. In the environment, the blocks will be arranged in columns, spanning from left to right. Keep in mind that some of these columns can be empty with no blocks currently placed in them. Within a column one or multiple blocks of different colors can be stacked on top of each other. Your task is to correctly evaluate the question based on the image provided.

Available actions
You have only one action available, called 'moveblock(block, column)'. This action allows you to move a block from its current column to the specified column. In order to perform this action, the block you want to move must be the topmost block of its column and must not already be in the target column. If the action is valid, the block will be moved to the specified column and will be placed on top of any blocks that are already in that column, if any.
To refer to the blocks, use lowercase letters for the colors: 'r' for red, 'g' for green, 'b' for blue, 'y' for yellow, 'p' for purple, 'o' for orange. To refer to the columns, use the labels provided in the image, 'c1', 'c2', 'c3', 'c4' and 'c5'.

Goal
{goal_string}

Previously taken actions

{previous_actions}

Current environment state
{image}
</user>

Plan prompt for ViPlan-HH

<system>
You are an expert planning assistant. You will be given an image which represents the current state of the environment you are in, a natural language description of the goal that needs to be achieved and a set of actions that can be performed in the environment. Your task is to generate a plan that achieves the goal, in the form of a sequence of actions that need to be executed to reach the goal. The format of your output should be a JSON object with the following structure:

```json
{
  "plan": [
    {
      "action": action_name,
      "parameters": ['parameter1', 'parameter2', ...]
    },
    ... other actions ...
    ]
}
```

You will also receive feedback of the previously taken actions, with a note showing if they failed or not. If an action failed, think about why that could be and then output a new plan accordingly.
</system>
<user>

Description of the environment
The environment is a virtual household simulator, with objects and furniture which can be interacted with. Keep in mind that some objects might not be visible or immediately reachable, in which case you need to navigate to them first. If after navigating to an object it is still not reachable, you might need to open a container.

Additional information
{privileged_info}

Available actions

- Action: grasp
  - Parameters:
    1. a movable object
  - Preconditions:
    - The object is within reach.
    - The agent is not holding anything.
  - Effects:
    - The agent picks up that object.
    - It is no longer on top of or next to any other object.
    - If it was inside a container, it leaves the container.

- Action: place-on
  - Parameters:
    1. the movable object being held
    2. another object to serve as support
  - Preconditions:
    - The agent is holding the first object.
    - The support object is within reach.
  - Effects:
    - The held object is placed on top of the support object.
    - The agent's hands become free.

- Action: place-next-to
  - Parameters:
    1. the movable object being held
    2. another object to stand beside
  - Preconditions:
    - The agent is holding the first object.
    - The other object is within reach.

  - Effects:
    - The held object is positioned next to the other object.
    - The agent's hands become free.

- Action: place-inside
  - Parameters:
    1. the movable object being held
    2. an open container
  - Preconditions:
    - The agent is holding the object.
    - The container is open and within reach.
  - Effects:
    - The object is placed inside the container.
    - The agent's hands become free.

- Action: open-container
  - Parameters:
    1. a closed container
  - Preconditions:
    - The container is within reach.
    - The agent is not holding anything.
  - Effects:
    - The container becomes open.
    - All objects inside it become reachable.

- Action: close-container
  - Parameters:
    1. an open container
  - Preconditions:
    - The container is within reach.
  - Effects:
    - The container becomes closed.
    - All objects inside it become unreachable.

- Action: navigate-to
  - Parameters:
    1. any target object
  - Preconditions:
    - The target object is currently out of reach and not hidden in a closed container.
  - Effects:
    - The target object becomes reachable.
    - All other objects become out of reach.
    - If the target is an open container, everything inside it also becomes reachable.

Goal
{goal_string}

Previously taken actions
{previous_actions}

Current environment state
{image}
</user>

Plan + CoT prompt for ViPlan-HH

<system>
You are an expert planning assistant. You will be given an image which represents the current state of the environment you are in, a natural language description of the goal that needs to be achieved and a set of actions that can be performed in the environment. Your task is to generate a plan that achieves the goal, in the form of a sequence of actions that need to be executed to reach the goal. Before answering with the plan, think carefully step by step about the actions you need to take and what the expected outcome of each action is. Write the reasoning behind the plan and justify each action you are going to take. Make sure that each action is possible, and if previous actions failed, reason about why this could be the case.

The format of your output should be a JSON object with the following structure. Make sure that the explanation is also written inside the json.

```json
{
  "explanation": <a detailed explanation of the plan>,
  "plan": [
    {
        "action": action_name,
        "parameters": ['parameter1', 'parameter2', ...]
    },
    ... other actions ...
    ]
}
```

You will also receive feedback of the previously taken actions, with a note showing if they failed or not. If an action failed, think about why that could be and then output a new plan accordingly.
</system>
<user>

Description of the environment
The environment is a virtual household simulator, with objects and furniture which can be interacted with. Keep in mind that some objects might not be visible or immediately reachable, in which case you need to navigate to them first. If after navigating to an object it is still not reachable, you might need to open a container.

Additional information
{privileged_info}

Available actions

- Action: grasp
  - Parameters:
    1. a movable object
  - Preconditions:
    - The object is within reach.
    - The agent is not holding anything.
  - Effects:
    - The agent picks up that object.
    - It is no longer on top of or next to any other object.
    - If it was inside a container, it leaves the container.

- Action: place-on
  - Parameters:
    1. the movable object being held
    2. another object to serve as support
  - Preconditions:
    - The agent is holding the first object.
    - The support object is within reach.
  - Effects:
    - The held object is placed on top of the support object.
    - The agent's hands become free.

- Action: place-next-to

  – Parameters:
    1. the movable object being held
    2. another object to stand beside
  – Preconditions:
    – The agent is holding the first object.
    – The other object is within reach.
  – Effects:
    – The held object is positioned next to the other object.
    – The agent's hands become free.

– Action: place–inside
  – Parameters:
    1. the movable object being held
    2. an open container
  – Preconditions:
    – The agent is holding the object.
    – The container is open and within reach.
  – Effects:
    – The object is placed inside the container.
    – The agent's hands become free.

– Action: open–container
  – Parameters:
    1. a closed container
  – Preconditions:
    – The container is within reach.
    – The agent is not holding anything.
  – Effects:
    – The container becomes open.
    – All objects inside it become reachable.

– Action: close–container
  – Parameters:
    1. an open container
  – Preconditions:
    – The container is within reach.
  – Effects:
    – The container becomes closed.
    – All objects inside it become unreachable.

– Action: navigate–to
  – Parameters:
    1. any target object
  – Preconditions:
    – The target object is currently out of reach and not hidden in a closed container.
  – Effects:
    – The target object becomes reachable.
    – All other objects become out of reach.
    – If the target is an open container, everything inside it also becomes reachable.

Goal
{goal_string}

Previously taken actions
{previous_actions}

Current environment state
{image}
</user>

Memory Context for Ground + Mem Variants

Previously, in the same state (image) as the one shown, you were asked these questions and provided the following answers:

- Q: {question}
  A: {answer}
  (repeated for all relevant Q/A pairs)

(the next line varies based on the previous outcome)
(if the previously selected action was illegal, but the VLM thought it was legal:)
Based on these answers, the action '{action_str}' was attempted, but something went wrong. This is very likely to have been caused by one or more incorrect answers above. Keep this in mind while answering the following question.

(if the previous action was deemed illegal by the VLM due to unsatisfied preconditions, but there is a chance it could have been legal:)
Based on these answers, the previously−planned action '{action_str}' was called off, as at least one of its preconditions was judged invalid. This might have been the correct choice or a mistake. Keep this in mind while answering the following question.

(if an unobserved precondition was not satisfied:)
Based on these answers, the action '{action_str}' was attempted, but something went wrong. This may have been due to an unobserved precondition not being met. Keep this in mind while answering the following question.

J Complete Selection Results

We report the individual success rate and accuracy for each model, method, domain and split in Tables 6, 7, 8, 9, 10, 11, 12 and 13. The task success rates for the full selection results are also visualized in Figure 15. Besides task success rate (which we use as our main metric), we further report predicate accuracy for Ground variants, computed as the fraction of yes-no questions that were answered correctly when compared to the ground truth environment. In practice, we observe that models generally show very high accuracy ($\geq 90\%$ for many model families); however, this does not always translate to high task success rate. This is due to compounding errors preventing successful completions: especially as the complexity grows, one episode requires up to 120 questions to be answered correctly, so even very accurate models can make a single mistake that compromises the task, as shown in Figure 16.

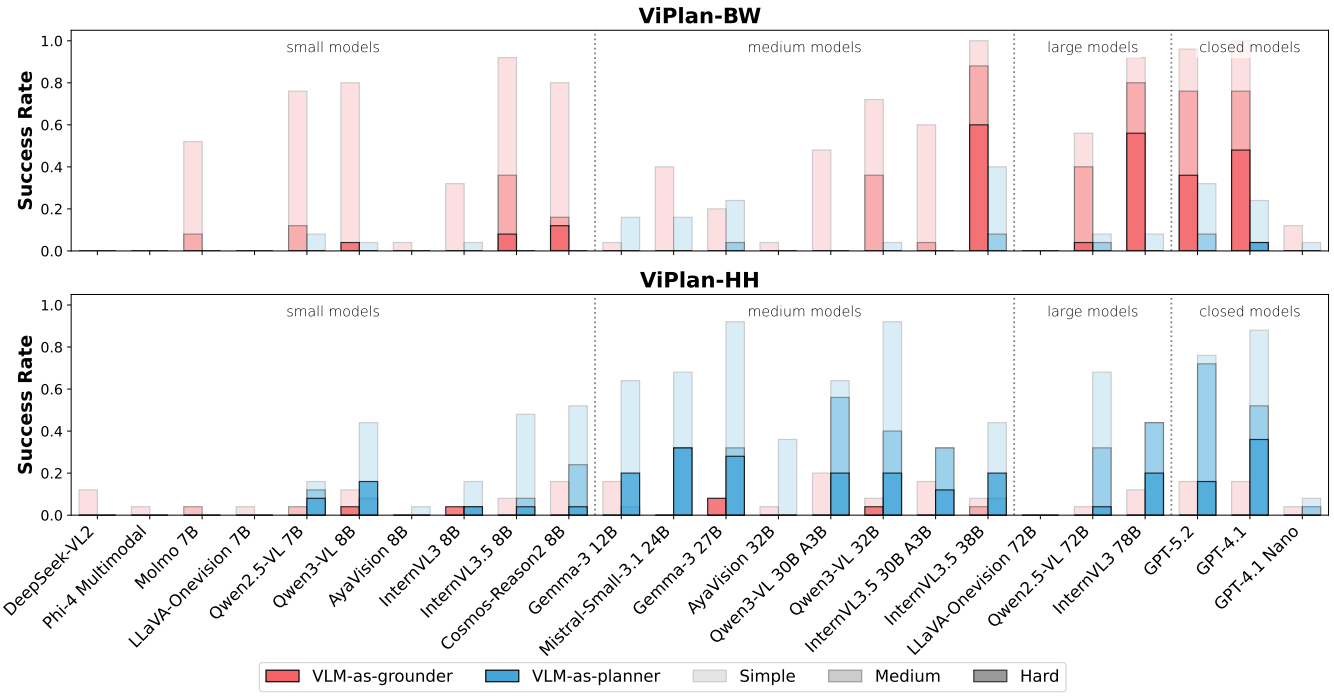


Figure 15: **Expanded Selection Results.** The Ground approach excels in ViPlan-BW (top), where GPT-4.1, InternVL3 78B and InternVL 3.5 38B complete a significant fraction of tasks, matching the performance of GPT-5.2 and GPT-4.1. The Plan approach is instead better on ViPlan-HH (bottom), where medium, large and closed models perform generally better than with Ground, with Gemma-3 27B and Qwen3-VL 32B standing out as medium-sized models.

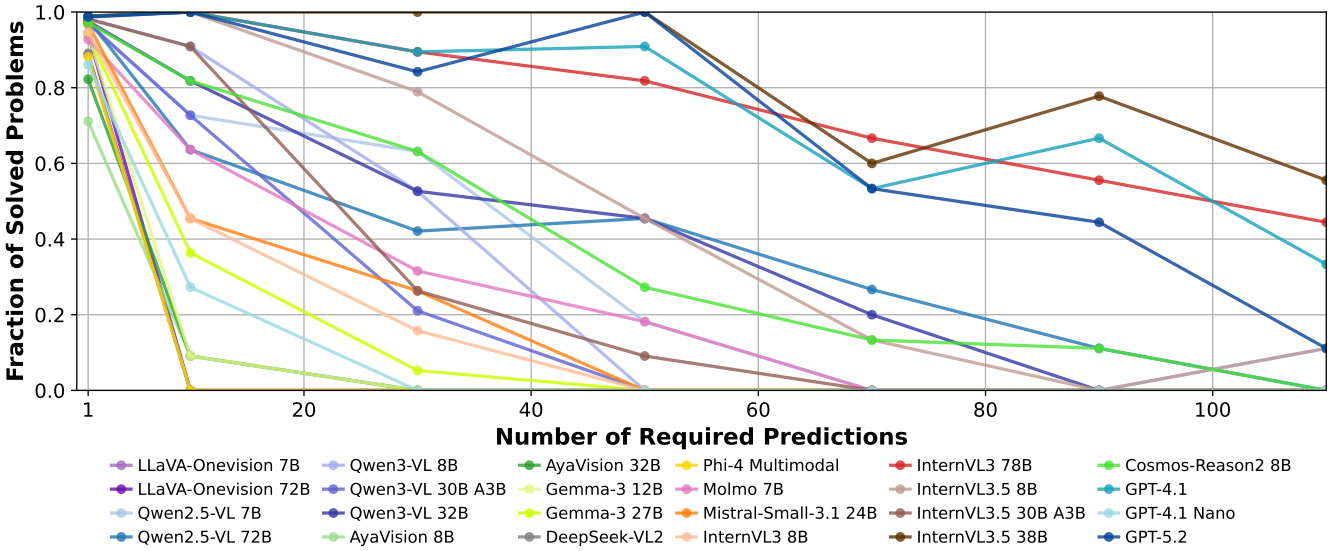


Figure 16: **Compounding Errors in Planning.** Analysis of the fractions of tasks solved in ViPlan-BW (all difficulties), based on the number of predictions a model would need to answer correctly to succeed for the Ground method. Benchmarks that ask independent questions to VLMs and measure their accuracy do not capture the effect that compounding errors have on solving a problem, and would correspond to measuring only one prediction. Most models score well on the single prediction, but deteriorate quickly as the errors compound.

Table 6: **Full Results for Ground and Ground + CoT on ViPlan-BW.** Success rate indicates the fraction of problems solved, accuracy is the fraction of correct predictions for a single predicate. The result is an average over the 25 problems in each split. Standard error of the mean is reported in parenthesis. The highest value in each column is bolded.

Model	CoT	Simple		Medium		Hard	
		Success	Accuracy	Success	Accuracy	Success	Accuracy
AyaVision 32B	×	0.04 (0.04)	0.88 (0.01)	0.00 (0.00)	0.82 (0.00)	0.00 (0.00)	0.76 (0.01)
AyaVision 32B	✓	0.00 (0.00)	0.89 (0.01)	0.04 (0.04)	0.85 (0.00)	0.00 (0.00)	0.79 (0.00)
AyaVision 8B	×	0.04 (0.04)	0.68 (0.01)	0.00 (0.00)	0.69 (0.01)	0.00 (0.00)	0.76 (0.01)
AyaVision 8B	✓	0.00 (0.00)	0.51 (0.01)	0.00 (0.00)	0.53 (0.01)	0.00 (0.00)	0.60 (0.01)
Cosmos-Reason2 8B	×	0.80 (0.08)	0.99 (0.00)	0.16 (0.07)	0.97 (0.00)	0.12 (0.06)	0.96 (0.00)
Cosmos-Reason2 8B	✓	0.44 (0.10)	0.97 (0.00)	0.12 (0.06)	0.98 (0.00)	0.04 (0.04)	0.96 (0.00)
DeepSeek-VL2	×	0.00 (0.00)	0.91 (0.01)	0.00 (0.00)	0.88 (0.01)	0.00 (0.00)	0.88 (0.01)
DeepSeek-VL2	✓	0.04 (0.04)	0.91 (0.00)	0.00 (0.00)	0.85 (0.00)	0.00 (0.00)	0.86 (0.00)
GPT-4.1	×	**1.00 (0.00)**	**1.00 (0.00)**	0.76 (0.09)	**0.99 (0.00)**	0.48 (0.10)	**0.98 (0.00)**
GPT-4.1	✓	0.92 (0.05)	0.99 (0.00)	0.76 (0.09)	0.98 (0.00)	0.44 (0.10)	**0.98 (0.00)**
GPT-4.1 Nano	×	0.12 (0.06)	0.89 (0.01)	0.00 (0.00)	0.85 (0.00)	0.00 (0.00)	0.84 (0.00)
GPT-4.1 Nano	✓	0.36 (0.10)	0.96 (0.00)	0.00 (0.00)	0.96 (0.00)	0.00 (0.00)	0.92 (0.00)
GPT-5.2	×	0.96 (0.04)	0.99 (0.00)	0.76 (0.09)	**0.99 (0.00)**	0.36 (0.10)	**0.98 (0.00)**
Gemma-3 12B	×	0.04 (0.04)	0.96 (0.01)	0.00 (0.00)	0.95 (0.00)	0.00 (0.00)	0.91 (0.00)
Gemma-3 12B	✓	0.16 (0.07)	0.93 (0.00)	0.00 (0.00)	0.91 (0.01)	0.00 (0.00)	0.88 (0.01)
Gemma-3 27B	×	0.20 (0.08)	0.96 (0.00)	0.00 (0.00)	0.93 (0.00)	0.00 (0.00)	0.93 (0.01)
Gemma-3 27B	✓	0.28 (0.09)	0.96 (0.00)	0.04 (0.04)	0.94 (0.00)	0.00 (0.00)	0.91 (0.00)
InternVL3 78B	×	0.92 (0.05)	**1.00 (0.00)**	0.80 (0.08)	**0.99 (0.00)**	0.56 (0.10)	**0.98 (0.00)**
InternVL3 78B	✓	0.96 (0.04)	**1.00 (0.00)**	0.80 (0.08)	**0.99 (0.00)**	0.56 (0.10)	**0.98 (0.00)**
InternVL3 8B	×	0.32 (0.09)	0.97 (0.00)	0.00 (0.00)	0.95 (0.00)	0.00 (0.00)	0.92 (0.00)
InternVL3 8B	✓	0.08 (0.05)	0.95 (0.00)	0.00 (0.00)	0.92 (0.00)	0.00 (0.00)	0.86 (0.00)
InternVL3.5 30B A3B	×	0.60 (0.10)	0.99 (0.00)	0.04 (0.04)	0.98 (0.00)	0.00 (0.00)	0.97 (0.00)
InternVL3.5 30B A3B	✓	0.80 (0.08)	0.99 (0.00)	0.24 (0.09)	0.97 (0.00)	0.00 (0.00)	0.94 (0.00)
InternVL3.5 38B	×	**1.00 (0.00)**	**1.00 (0.00)**	**0.88 (0.06)**	**0.99 (0.00)**	**0.60 (0.10)**	**0.98 (0.00)**
InternVL3.5 38B	✓	**1.00 (0.00)**	**1.00 (0.00)**	**0.88 (0.06)**	0.98 (0.00)	0.36 (0.10)	0.95 (0.00)
InternVL3.5 8B	×	0.92 (0.05)	0.99 (0.00)	0.36 (0.10)	**0.99 (0.00)**	0.08 (0.05)	**0.98 (0.00)**
InternVL3.5 8B	✓	0.88 (0.06)	0.99 (0.00)	0.40 (0.10)	0.98 (0.00)	0.20 (0.08)	0.97 (0.00)
LLaVA-Onevision 72B	×	0.00 (0.00)	0.95 (0.01)	0.00 (0.00)	0.94 (0.00)	0.00 (0.00)	0.93 (0.01)
LLaVA-Onevision 72B	✓	0.04 (0.04)	0.96 (0.00)	0.00 (0.00)	0.94 (0.00)	0.00 (0.00)	0.94 (0.00)
LLaVA-Onevision 7B	×	0.00 (0.00)	0.92 (0.01)	0.00 (0.00)	0.88 (0.01)	0.00 (0.00)	0.86 (0.01)
LLaVA-Onevision 7B	✓	0.00 (0.00)	0.93 (0.01)	0.00 (0.00)	0.88 (0.01)	0.00 (0.00)	0.89 (0.01)
Mistral-Small-3.1 24B	×	0.40 (0.10)	0.99 (0.00)	0.00 (0.00)	0.96 (0.00)	0.00 (0.00)	0.96 (0.00)
Mistral-Small-3.1 24B	✓	0.56 (0.10)	0.98 (0.00)	0.04 (0.04)	0.95 (0.00)	0.00 (0.00)	0.94 (0.00)
Molmo 7B	×	0.52 (0.10)	0.94 (0.00)	0.08 (0.05)	0.93 (0.00)	0.00 (0.00)	0.91 (0.00)
Molmo 7B	✓	0.00 (0.00)	0.77 (0.01)	0.00 (0.00)	0.75 (0.01)	0.00 (0.00)	0.74 (0.01)
Phi-4 Multimodal	×	0.00 (0.00)	0.92 (0.01)	0.00 (0.00)	0.89 (0.01)	0.00 (0.00)	0.83 (0.01)
Phi-4 Multimodal	✓	0.00 (0.00)	0.08 (0.01)	0.00 (0.00)	0.15 (0.01)	0.00 (0.00)	0.19 (0.01)
Qwen2.5-VL 72B	×	0.56 (0.10)	0.98 (0.00)	0.40 (0.10)	**0.99 (0.00)**	0.04 (0.04)	0.97 (0.00)
Qwen2.5-VL 72B	✓	0.48 (0.10)	0.97 (0.00)	0.20 (0.08)	**0.99 (0.00)**	0.00 (0.00)	0.96 (0.00)
Qwen2.5-VL 7B	×	0.76 (0.09)	0.99 (0.00)	0.12 (0.06)	0.97 (0.00)	0.00 (0.00)	0.94 (0.00)
Qwen2.5-VL 7B	✓	0.08 (0.05)	0.98 (0.00)	0.00 (0.00)	0.92 (0.00)	0.00 (0.00)	0.89 (0.00)
Qwen3-VL 30B A3B	×	0.48 (0.10)	0.98 (0.00)	0.00 (0.00)	0.97 (0.00)	0.00 (0.00)	0.96 (0.00)
Qwen3-VL 30B A3B	✓	0.60 (0.10)	0.97 (0.00)	0.20 (0.08)	0.97 (0.00)	0.12 (0.06)	0.97 (0.00)
Qwen3-VL 32B	×	0.72 (0.09)	0.98 (0.00)	0.36 (0.10)	0.98 (0.00)	0.00 (0.00)	0.96 (0.00)
Qwen3-VL 32B	✓	0.56 (0.10)	0.97 (0.00)	0.44 (0.10)	0.98 (0.00)	0.04 (0.04)	0.97 (0.00)
Qwen3-VL 8B	×	0.80 (0.08)	0.99 (0.00)	0.00 (0.00)	0.98 (0.00)	0.04 (0.04)	**0.98 (0.00)**
Qwen3-VL 8B	✓	0.80 (0.08)	0.99 (0.00)	0.40 (0.10)	0.98 (0.00)	0.40 (0.10)	0.96 (0.00)

Table 7: **Full Results for Ground + Mem and Ground + Mem + CoT on ViPlan-BW.** Success rate indicates the fraction of problems solved, accuracy is the fraction of correct predictions. The highest value in each column is bolded.

Model	CoT	Simple		Medium		Hard	
		Success	Accuracy	Success	Accuracy	Success	Accuracy
Gemma-3 27B	×	0.24 (0.09)	0.96 (0.00)	0.00 (0.00)	0.94 (0.00)	0.00 (0.00)	0.93 (0.00)
Gemma-3 27B	✓	0.28 (0.09)	0.96 (0.00)	0.00 (0.00)	0.93 (0.00)	0.00 (0.00)	0.92 (0.00)
InternVL3 78B	×	0.92 (0.05)	**1.00 (0.00)**	0.80 (0.08)	**0.99 (0.00)**	**0.68 (0.09)**	**0.98 (0.00)**
InternVL3 78B	✓	0.96 (0.04)	0.99 (0.00)	0.76 (0.09)	**0.99 (0.00)**	0.64 (0.10)	**0.98 (0.00)**
InternVL3.5 38B	×	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**	**0.99 (0.00)**	0.64 (0.10)	**0.98 (0.00)**
InternVL3.5 38B	✓	**1.00 (0.00)**	0.99 (0.00)	0.84 (0.07)	**0.99 (0.00)**	0.48 (0.10)	0.96 (0.00)
Qwen2.5-VL 72B	×	0.60 (0.10)	0.98 (0.00)	0.20 (0.08)	0.98 (0.00)	0.00 (0.00)	0.96 (0.00)
Qwen2.5-VL 72B	✓	0.52 (0.10)	0.97 (0.00)	0.16 (0.07)	0.98 (0.00)	0.00 (0.00)	0.96 (0.00)
Qwen3-VL 32B	×	0.64 (0.10)	0.99 (0.00)	0.32 (0.09)	0.98 (0.00)	0.04 (0.04)	0.97 (0.00)
Qwen3-VL 32B	✓	0.56 (0.10)	0.98 (0.00)	0.48 (0.10)	0.98 (0.00)	0.08 (0.05)	0.97 (0.00)

Table 8: **Full Results for Ground + Mem and Ground + Mem + CoT on ViPlan-HH.** Success rate indicates the fraction of problems solved, accuracy is the fraction of correct predictions. The highest value in each column is bolded.

Model	CoT	Simple		Medium		Hard	
		Success	Accuracy	Success	Accuracy	Success	Accuracy
Gemma-3 27B	×	0.00 (0.00)	0.69 (0.02)	0.00 (0.00)	**0.79 (0.00)**	0.00 (0.00)	0.76 (0.00)
Gemma-3 27B	✓	0.12 (0.06)	0.62 (0.01)	0.00 (0.00)	0.67 (0.00)	0.00 (0.00)	0.66 (0.00)
InternVL3 78B	×	0.12 (0.06)	0.66 (0.01)	**0.04 (0.04)**	0.63 (0.01)	0.04 (0.04)	0.59 (0.00)
InternVL3 78B	✓	0.16 (0.07)	0.64 (0.01)	0.00 (0.00)	0.71 (0.00)	0.00 (0.00)	0.67 (0.00)
InternVL3.5 38B	×	**0.20 (0.08)**	0.59 (0.01)	0.00 (0.00)	0.62 (0.01)	0.04 (0.04)	0.71 (0.00)
InternVL3.5 38B	✓	0.12 (0.06)	0.64 (0.01)	0.00 (0.00)	0.68 (0.01)	**0.08 (0.05)**	0.68 (0.00)
Qwen2.5-VL 72B	×	0.04 (0.04)	0.66 (0.01)	0.00 (0.00)	**0.79 (0.00)**	0.00 (0.00)	**0.77 (0.00)**
Qwen2.5-VL 72B	✓	0.04 (0.04)	**0.72 (0.01)**	0.00 (0.00)	0.77 (0.00)	0.00 (0.00)	0.71 (0.00)
Qwen3-VL 32B	×	**0.20 (0.08)**	**0.72 (0.01)**	**0.04 (0.04)**	0.74 (0.01)	0.00 (0.00)	0.74 (0.00)
Qwen3-VL 32B	✓	0.12 (0.06)	0.68 (0.01)	0.00 (0.00)	0.70 (0.01)	0.04 (0.04)	0.64 (0.00)

Table 9: **Full Results for Plan and Plan + CoT. on ViPlan-BW.** Success rate indicates the fraction of problems solved. The result is an average over the 25 problems in each split. Standard error of the mean is reported in parenthesis. The highest value in each column is bolded.

Model	CoT	Simple	Medium	Hard
AyaVision 32B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
AyaVision 32B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
AyaVision 8B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
AyaVision 8B	✓	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
Cosmos-Reason2 8B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Cosmos-Reason2 8B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
DeepSeek-VL2	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
DeepSeek-VL2	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
GPT-4.1	×	0.24 (0.09)	0.04 (0.04)	0.04 (0.04)
GPT-4.1	✓	**0.84 (0.07)**	**0.48 (0.10)**	**0.12 (0.06)**
GPT-4.1 Nano	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
GPT-4.1 Nano	✓	0.24 (0.09)	0.00 (0.00)	0.00 (0.00)
GPT-5.2	×	0.32 (0.09)	0.08 (0.05)	0.00 (0.00)
Gemma-3 12B	×	0.16 (0.07)	0.00 (0.00)	0.00 (0.00)
Gemma-3 12B	✓	0.28 (0.09)	0.00 (0.00)	0.00 (0.00)
Gemma-3 27B	×	0.24 (0.09)	0.04 (0.04)	0.00 (0.00)
Gemma-3 27B	✓	0.20 (0.08)	0.12 (0.06)	0.04 (0.04)
InternVL3 78B	×	0.08 (0.05)	0.00 (0.00)	0.00 (0.00)
InternVL3 78B	✓	0.12 (0.06)	0.00 (0.00)	0.00 (0.00)
InternVL3 8B	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
InternVL3 8B	✓	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 30B A3B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 30B A3B	✓	0.08 (0.05)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 38B	×	0.40 (0.10)	0.08 (0.05)	0.00 (0.00)
InternVL3.5 38B	✓	0.12 (0.06)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 8B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 8B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 72B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 72B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 7B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 7B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Mistral-Small-3.1 24B	×	0.16 (0.07)	0.00 (0.00)	0.00 (0.00)
Mistral-Small-3.1 24B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Molmo 7B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Molmo 7B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Phi-4 Multimodal	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Phi-4 Multimodal	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Qwen2.5-VL 72B	×	0.08 (0.05)	0.04 (0.04)	0.00 (0.00)
Qwen2.5-VL 72B	✓	0.16 (0.07)	0.04 (0.04)	0.00 (0.00)
Qwen2.5-VL 7B	×	0.08 (0.05)	0.00 (0.00)	0.00 (0.00)
Qwen2.5-VL 7B	✓	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 30B A3B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 30B A3B	✓	0.20 (0.08)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 32B	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 32B	✓	0.48 (0.10)	0.20 (0.08)	0.04 (0.04)
Qwen3-VL 8B	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 8B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)

Table 10: **Full Results for Ground and Ground + CoT on ViPlan-HH.** Success rate indicates the fraction of problems solved, accuracy is the fraction of correct predictions for a single predicate. The result is an average over the 25 problems in each split. Standard error of the mean is reported in parenthesis. The highest value in each column is bolded.

Model	CoT	Simple		Medium		Hard	
		Success	Accuracy	Success	Accuracy	Success	Accuracy
AyaVision 32B	×	0.04 (0.04)	0.61 (0.02)	0.00 (0.00)	0.65 (0.01)	0.00 (0.00)	0.65 (0.01)
AyaVision 32B	✓	0.04 (0.04)	0.72 (0.01)	0.00 (0.00)	0.75 (0.00)	0.00 (0.00)	0.74 (0.01)
AyaVision 8B	×	0.00 (0.00)	0.46 (0.02)	0.00 (0.00)	0.44 (0.01)	0.00 (0.00)	0.44 (0.01)
AyaVision 8B	✓	0.00 (0.00)	0.67 (0.01)	0.00 (0.00)	0.70 (0.01)	0.00 (0.00)	0.69 (0.00)
Cosmos-Reason2 8B	×	0.16 (0.07)	0.57 (0.01)	0.00 (0.00)	0.65 (0.01)	0.00 (0.00)	0.50 (0.01)
Cosmos-Reason2 8B	✓	0.00 (0.00)	0.69 (0.01)	0.00 (0.00)	0.72 (0.01)	0.00 (0.00)	0.73 (0.01)
DeepSeek-VL2	×	0.12 (0.06)	0.71 (0.01)	0.00 (0.00)	0.75 (0.00)	0.00 (0.00)	0.75 (0.01)
DeepSeek-VL2	✓	0.00 (0.00)	0.69 (0.01)	0.00 (0.00)	0.70 (0.00)	0.00 (0.00)	0.68 (0.01)
GPT-4.1	×	0.16 (0.07)	0.68 (0.01)	0.00 (0.00)	0.74 (0.01)	0.00 (0.00)	0.70 (0.01)
GPT-4.1	✓	0.20 (0.08)	0.69 (0.01)	0.04 (0.04)	0.72 (0.01)	0.00 (0.00)	0.71 (0.00)
GPT-4.1 Nano	×	0.04 (0.04)	0.54 (0.01)	0.00 (0.00)	0.60 (0.01)	0.00 (0.00)	0.53 (0.01)
GPT-4.1 Nano	✓	0.04 (0.04)	0.67 (0.01)	0.00 (0.00)	0.72 (0.01)	0.00 (0.00)	0.66 (0.01)
GPT-5.2	×	0.16 (0.07)	0.65 (0.01)	0.00 (0.00)	0.76 (0.00)	0.00 (0.00)	0.82 (0.00)
Gemma-3 12B	×	0.16 (0.07)	0.52 (0.02)	0.00 (0.00)	0.68 (0.01)	0.00 (0.00)	0.59 (0.01)
Gemma-3 12B	✓	0.12 (0.06)	0.65 (0.01)	0.00 (0.00)	0.58 (0.01)	0.00 (0.00)	0.60 (0.01)
Gemma-3 27B	×	0.08 (0.05)	0.67 (0.01)	0.00 (0.00)	0.77 (0.00)	0.08 (0.05)	0.75 (0.00)
Gemma-3 27B	✓	0.12 (0.06)	0.70 (0.01)	0.00 (0.00)	0.68 (0.01)	0.00 (0.00)	0.66 (0.00)
InternVL3 78B	×	0.12 (0.06)	0.71 (0.01)	0.00 (0.00)	0.68 (0.01)	0.00 (0.00)	0.65 (0.01)
InternVL3 78B	✓	**0.28 (0.09)**	0.66 (0.01)	0.00 (0.00)	0.69 (0.01)	0.04 (0.04)	0.70 (0.01)
InternVL3 8B	×	0.00 (0.00)	0.73 (0.01)	0.00 (0.00)	0.78 (0.00)	0.04 (0.04)	0.78 (0.01)
InternVL3 8B	✓	0.08 (0.05)	0.62 (0.01)	0.00 (0.00)	0.59 (0.01)	0.00 (0.00)	0.56 (0.01)
InternVL3.5 30B A3B	×	0.16 (0.07)	0.68 (0.01)	0.00 (0.00)	0.73 (0.01)	0.00 (0.00)	0.68 (0.01)
InternVL3.5 30B A3B	✓	0.04 (0.04)	0.72 (0.01)	0.00 (0.00)	0.75 (0.01)	0.00 (0.00)	0.76 (0.01)
InternVL3.5 38B	×	0.08 (0.05)	0.52 (0.01)	0.04 (0.04)	0.55 (0.01)	0.00 (0.00)	0.46 (0.01)
InternVL3.5 38B	✓	0.08 (0.05)	0.68 (0.01)	0.00 (0.00)	0.66 (0.01)	**0.12 (0.06)**	0.69 (0.00)
InternVL3.5 8B	×	0.08 (0.05)	0.69 (0.01)	0.00 (0.00)	0.72 (0.01)	0.00 (0.00)	0.68 (0.01)
InternVL3.5 8B	✓	0.04 (0.04)	0.65 (0.01)	0.00 (0.00)	0.70 (0.01)	0.00 (0.00)	0.71 (0.01)
LLaVA-Onevision 72B	×	0.00 (0.00)	0.64 (0.02)	0.00 (0.00)	0.80 (0.01)	0.00 (0.00)	0.75 (0.01)
LLaVA-Onevision 72B	✓	0.00 (0.00)	0.69 (0.02)	0.04 (0.04)	0.79 (0.01)	0.00 (0.00)	0.78 (0.02)
LLaVA-Onevision 7B	×	0.04 (0.04)	0.59 (0.01)	0.00 (0.00)	0.68 (0.01)	0.00 (0.00)	0.69 (0.01)
LLaVA-Onevision 7B	✓	0.04 (0.04)	0.58 (0.01)	0.00 (0.00)	0.65 (0.01)	0.00 (0.00)	0.70 (0.01)
Mistral-Small-3.1 24B	×	0.00 (0.00)	0.69 (0.01)	0.00 (0.00)	**0.83 (0.00)**	0.00 (0.00)	**0.83 (0.00)**
Mistral-Small-3.1 24B	✓	0.04 (0.04)	0.60 (0.02)	0.00 (0.00)	0.64 (0.01)	0.00 (0.00)	0.67 (0.01)
Molmo 7B	×	0.00 (0.00)	0.60 (0.02)	0.04 (0.04)	0.65 (0.01)	0.00 (0.00)	0.67 (0.00)
Molmo 7B	✓	0.00 (0.00)	0.68 (0.01)	0.00 (0.00)	0.71 (0.01)	0.00 (0.00)	0.78 (0.00)
Phi-4 Multimodal	×	0.04 (0.04)	0.31 (0.02)	0.00 (0.00)	0.22 (0.01)	0.00 (0.00)	0.33 (0.01)
Phi-4 Multimodal	✓	0.00 (0.00)	0.05 (0.03)	0.00 (0.00)	0.02 (0.00)	0.00 (0.00)	0.17 (0.02)
Qwen2.5-VL 72B	×	0.04 (0.04)	0.72 (0.01)	0.00 (0.00)	0.80 (0.00)	0.00 (0.00)	0.78 (0.00)
Qwen2.5-VL 72B	✓	0.00 (0.00)	0.70 (0.01)	0.00 (0.00)	0.75 (0.01)	0.00 (0.00)	0.76 (0.00)
Qwen2.5-VL 7B	×	0.00 (0.00)	0.73 (0.01)	0.04 (0.04)	0.81 (0.00)	0.00 (0.00)	0.82 (0.00)
Qwen2.5-VL 7B	✓	0.04 (0.04)	0.68 (0.01)	0.00 (0.00)	0.73 (0.00)	0.04 (0.04)	0.62 (0.00)
Qwen3-VL 30B A3B	×	0.20 (0.08)	0.75 (0.01)	0.00 (0.00)	0.72 (0.01)	0.00 (0.00)	0.71 (0.01)
Qwen3-VL 30B A3B	✓	0.08 (0.05)	0.68 (0.01)	0.00 (0.00)	0.74 (0.01)	0.00 (0.00)	0.73 (0.00)
Qwen3-VL 32B	×	0.08 (0.05)	0.71 (0.01)	0.00 (0.00)	0.71 (0.01)	0.04 (0.04)	0.66 (0.01)
Qwen3-VL 32B	✓	0.12 (0.06)	0.66 (0.01)	**0.08 (0.05)**	0.71 (0.01)	0.04 (0.04)	0.69 (0.00)
Qwen3-VL 8B	×	0.12 (0.06)	0.66 (0.01)	0.00 (0.00)	0.73 (0.01)	0.04 (0.04)	0.72 (0.01)
Qwen3-VL 8B	✓	0.04 (0.04)	**0.76 (0.01)**	0.00 (0.00)	0.72 (0.01)	0.00 (0.00)	0.74 (0.00)

Table 11: **Full Results for Plan and Plan + CoT on ViPlan-HH.** Success rate indicates the fraction of problems solved. The result is an average over the 25 problems in each split. Standard error of the mean is reported in parenthesis. The highest value in each column is bolded.

Model	CoT	Simple	Medium	Hard
AyaVision 32B	×	0.36 (0.10)	0.00 (0.00)	0.00 (0.00)
AyaVision 32B	✓	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
AyaVision 8B	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
AyaVision 8B	✓	0.00 (0.00)	0.00 (0.00)	0.04 (0.04)
Cosmos-Reason2 8B	×	0.52 (0.10)	0.24 (0.09)	0.04 (0.04)
Cosmos-Reason2 8B	✓	0.44 (0.10)	0.12 (0.06)	0.24 (0.09)
DeepSeek-VL2	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
DeepSeek-VL2	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
GPT-4.1	×	0.88 (0.06)	0.52 (0.10)	**0.36 (0.10)**
GPT-4.1	✓	0.60 (0.10)	0.44 (0.10)	0.32 (0.09)
GPT-4.1 Nano	×	0.08 (0.05)	0.04 (0.04)	0.00 (0.00)
GPT-4.1 Nano	✓	0.16 (0.07)	0.04 (0.04)	0.04 (0.04)
GPT-5.2	×	0.76 (0.09)	**0.72 (0.09)**	0.16 (0.07)
Gemma-3 12B	×	0.64 (0.10)	0.04 (0.04)	0.20 (0.08)
Gemma-3 12B	✓	0.48 (0.10)	0.00 (0.00)	0.00 (0.00)
Gemma-3 27B	×	**0.92 (0.05)**	0.32 (0.09)	0.28 (0.09)
Gemma-3 27B	✓	0.76 (0.09)	0.16 (0.07)	0.04 (0.04)
InternVL3 78B	×	0.44 (0.10)	0.44 (0.10)	0.20 (0.08)
InternVL3 78B	✓	0.36 (0.10)	0.04 (0.04)	0.08 (0.05)
InternVL3 8B	×	0.16 (0.07)	0.04 (0.04)	0.04 (0.04)
InternVL3 8B	✓	0.44 (0.10)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 30B A3B	×	0.32 (0.09)	0.32 (0.09)	0.12 (0.06)
InternVL3.5 30B A3B	✓	0.12 (0.06)	0.00 (0.00)	0.00 (0.00)
InternVL3.5 38B	×	0.44 (0.10)	0.08 (0.05)	0.20 (0.08)
InternVL3.5 38B	✓	0.32 (0.09)	0.12 (0.06)	0.00 (0.00)
InternVL3.5 8B	×	0.48 (0.10)	0.08 (0.05)	0.04 (0.04)
InternVL3.5 8B	✓	0.36 (0.10)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 72B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 72B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 7B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
LLaVA-Onevision 7B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Mistral-Small-3.1 24B	×	0.68 (0.09)	0.32 (0.09)	0.32 (0.09)
Mistral-Small-3.1 24B	✓	0.44 (0.10)	0.28 (0.09)	0.04 (0.04)
Molmo 7B	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Molmo 7B	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Phi-4 Multimodal	×	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Phi-4 Multimodal	✓	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)
Qwen2.5-VL 72B	×	0.68 (0.09)	0.32 (0.09)	0.04 (0.04)
Qwen2.5-VL 72B	✓	0.44 (0.10)	0.16 (0.07)	0.32 (0.09)
Qwen2.5-VL 7B	×	0.16 (0.07)	0.12 (0.06)	0.08 (0.05)
Qwen2.5-VL 7B	✓	0.24 (0.09)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 30B A3B	×	0.64 (0.10)	0.56 (0.10)	0.20 (0.08)
Qwen3-VL 30B A3B	✓	0.40 (0.10)	0.32 (0.09)	0.32 (0.09)
Qwen3-VL 32B	×	**0.92 (0.05)**	0.40 (0.10)	0.20 (0.08)
Qwen3-VL 32B	✓	0.40 (0.10)	0.12 (0.06)	0.08 (0.05)
Qwen3-VL 8B	×	0.44 (0.10)	0.08 (0.05)	0.16 (0.07)
Qwen3-VL 8B	✓	0.20 (0.08)	0.16 (0.07)	0.08 (0.05)

Table 12: **Full Results for Action and Action + CoT on ViPlan-BW.** Success rate indicates the fraction of problems solved. The highest value in each column is bolded.

Model	CoT	Simple	Medium	Hard
Gemma-3 27B	×	0.28 (0.09)	0.08 (0.05)	0.00 (0.00)
Gemma-3 27B	✓	0.28 (0.09)	0.08 (0.05)	**0.04 (0.04)**
InternVL3 78B	×	0.08 (0.05)	0.00 (0.00)	0.00 (0.00)
InternVL3 78B	✓	**0.56 (0.10)**	0.08 (0.05)	**0.04 (0.04)**
InternVL3.5 38B	×	0.48 (0.10)	0.04 (0.04)	0.00 (0.00)
InternVL3.5 38B	✓	0.12 (0.06)	0.04 (0.04)	0.00 (0.00)
Qwen2.5-VL 72B	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
Qwen2.5-VL 72B	✓	0.12 (0.06)	0.00 (0.00)	**0.04 (0.04)**
Qwen3-VL 32B	×	0.04 (0.04)	0.00 (0.00)	0.00 (0.00)
Qwen3-VL 32B	✓	0.40 (0.10)	**0.32 (0.09)**	0.00 (0.00)

Table 13: **Full Results for Action and Action + CoT on ViPlan-HH.** Success rate indicates the fraction of problems solved. The highest value in each column is bolded.

Model	CoT	Simple	Medium	Hard
Gemma-3 27B	×	**0.80 (0.08)**	0.20 (0.08)	0.32 (0.09)
Gemma-3 27B	✓	0.72 (0.09)	0.28 (0.09)	0.16 (0.07)
InternVL3 78B	×	0.44 (0.10)	**0.48 (0.10)**	0.32 (0.09)
InternVL3 78B	✓	0.36 (0.10)	0.32 (0.09)	0.16 (0.07)
InternVL3.5 38B	×	0.48 (0.10)	0.32 (0.09)	0.12 (0.06)
InternVL3.5 38B	✓	0.44 (0.10)	0.16 (0.07)	0.08 (0.05)
Qwen2.5-VL 72B	×	0.56 (0.10)	0.40 (0.10)	0.24 (0.09)
Qwen2.5-VL 72B	✓	0.60 (0.10)	0.36 (0.10)	**0.36 (0.10)**
Qwen3-VL 32B	×	0.76 (0.09)	0.44 (0.10)	0.28 (0.09)
Qwen3-VL 32B	✓	**0.80 (0.08)**	0.32 (0.09)	0.12 (0.06)

K Action Failures on ViPlan-BW

While the main ViPlan-BW experiments were performed without action failures, the possibility of actions going wrong in the real world is a motivating example particularly for the VLM-as-grounder evaluation setting, as verifying preconditions and effects is a natural way of counteracting action failures. We opted to exclude this from the main results as, with action failures, a task can fail either due to the VLMs' poor performance or simply due to poor luck, which in turn makes comparing different models difficult. However, to test the robustness to action failures of the approach, we report the results with a probability of action failure $p_f = 0.1$, which we implement into our simulator, in Table 14 for Ground and in Table 15 for Plan.
Overall, we observe similar trends as the version with no failures (Table 6), showing the resilience of the Ground setting, and validating the no-failure choice for the main benchmark.
Note that, as these experiments were performed in a preliminary version of the benchmark, some VLMs are missing from the results. Nevertheless, the method is the same, so the results are directly comparable to the ones in the main benchmark.

Table 14: **Full Results for Ground on ViPlan-BW with action failures.** Actions have a 10% chance of failing. Success rate indicates the fraction of problems solved, accuracy is the fraction of correct predictions for a single predicate. The result is an average over the 25 problems in each split. Standard error of the mean is reported in parenthesis. The highest value in each column is bolded.

Model	CoT	Simple		Medium		Hard	
		Success	Accuracy	Success	Accuracy	Success	Accuracy
AyaVision 32B	×	0.04 (0.04)	0.88 (0.01)	0.00 (0.00)	0.84 (0.00)	0.00 (0.00)	0.76 (0.00)
AyaVision 8B	×	0.04 (0.04)	0.68 (0.01)	0.00 (0.00)	0.70 (0.01)	0.00 (0.00)	0.76 (0.01)
Cosmos-Reason2 8B	×	0.84 (0.07)	0.99 (0.00)	0.00 (0.00)	0.97 (0.00)	0.08 (0.05)	0.96 (0.00)
DeepSeek-VL2	×	0.00 (0.00)	0.91 (0.01)	0.00 (0.00)	0.88 (0.01)	0.00 (0.00)	0.88 (0.01)
GPT-4.1 Nano	×	0.16 (0.07)	0.89 (0.01)	0.00 (0.00)	0.84 (0.00)	0.00 (0.00)	0.84 (0.00)
Gemma-3 12B	×	0.04 (0.04)	0.96 (0.00)	0.00 (0.00)	0.95 (0.00)	0.00 (0.00)	0.91 (0.00)
Gemma-3 27B	×	0.20 (0.08)	0.96 (0.00)	0.00 (0.00)	0.94 (0.00)	0.00 (0.00)	0.92 (0.01)
InternVL3 78B	×	0.92 (0.05)	0.99 (0.00)	0.84 (0.07)	**0.99 (0.00)**	**0.76 (0.09)**	**0.98 (0.00)**
InternVL3 8B	×	0.32 (0.09)	0.97 (0.00)	0.00 (0.00)	0.95 (0.00)	0.00 (0.00)	0.92 (0.00)
InternVL3.5 30B A3B	×	0.44 (0.10)	0.99 (0.00)	0.08 (0.05)	**0.99 (0.00)**	0.00 (0.00)	0.97 (0.00)
InternVL3.5 38B	×	**1.00 (0.00)**	**1.00 (0.00)**	**0.88 (0.06)**	**0.99 (0.00)**	0.68 (0.09)	**0.98 (0.00)**
InternVL3.5 8B	×	0.92 (0.05)	0.99 (0.00)	0.32 (0.09)	**0.99 (0.00)**	0.16 (0.07)	**0.98 (0.00)**
LLaVA-Onevision 72B	×	0.00 (0.00)	0.95 (0.01)	0.00 (0.00)	0.94 (0.00)	0.00 (0.00)	0.93 (0.00)
LLaVA-Onevision 7B	×	0.00 (0.00)	0.92 (0.01)	0.00 (0.00)	0.88 (0.01)	0.00 (0.00)	0.86 (0.01)
Mistral-Small-3.1 24B	×	0.36 (0.10)	0.99 (0.00)	0.00 (0.00)	0.96 (0.00)	0.00 (0.00)	0.95 (0.00)
Molmo 7B	×	0.44 (0.10)	0.94 (0.00)	0.04 (0.04)	0.93 (0.00)	0.08 (0.05)	0.92 (0.00)
Phi-4 Multimodal	×	0.00 (0.00)	0.92 (0.01)	0.00 (0.00)	0.90 (0.00)	0.00 (0.00)	0.83 (0.01)
Qwen2.5-VL 72B	×	0.52 (0.10)	0.98 (0.00)	0.44 (0.10)	**0.99 (0.00)**	0.08 (0.05)	0.97 (0.00)
Qwen2.5-VL 7B	×	0.76 (0.09)	0.98 (0.00)	0.12 (0.06)	0.97 (0.00)	0.00 (0.00)	0.95 (0.00)
Qwen3-VL 30B A3B	×	0.32 (0.09)	0.98 (0.00)	0.00 (0.00)	0.98 (0.00)	0.00 (0.00)	0.97 (0.00)
Qwen3-VL 8B	×	0.76 (0.09)	0.99 (0.00)	0.00 (0.00)	0.98 (0.00)	0.08 (0.05)	0.97 (0.00)

Table 15: **Full Results for Plan on ViPlan-BW with action failures.** Actions have a 10% chance of failing. Success rate indicates the fraction of problems solved. The result is an average over the 25 problems in each split. Standard error of the mean is reported in parenthesis. The highest value in each column is bolded.

Model	CoT	Simple	Medium	Hard
AyaVision 32B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
AyaVision 8B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
Cosmos-Reason2 8B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
DeepSeek-VL2	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
GPT-4.1 Nano	×	0.08 (0.05)	**0.00 (0.00)**	0.00 (0.00)
Gemma-3 12B	×	0.20 (0.08)	**0.00 (0.00)**	0.00 (0.00)
Gemma-3 27B	×	0.24 (0.09)	**0.00 (0.00)**	0.00 (0.00)
InternVL3 78B	×	0.08 (0.05)	**0.00 (0.00)**	0.00 (0.00)
InternVL3 8B	×	0.04 (0.04)	**0.00 (0.00)**	0.00 (0.00)
InternVL3.5 30B A3B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
InternVL3.5 38B	×	**0.32 (0.09)**	**0.00 (0.00)**	0.00 (0.00)
InternVL3.5 8B	×	0.00 (0.00)	**0.00 (0.00)**	**0.04 (0.04)**
LLaVA-Onevision 72B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
LLaVA-Onevision 7B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
Mistral-Small-3.1 24B	×	0.20 (0.08)	**0.00 (0.00)**	0.00 (0.00)
Molmo 7B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
Phi-4 Multimodal	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
Qwen2.5-VL 72B	×	0.08 (0.05)	**0.00 (0.00)**	0.00 (0.00)
Qwen2.5-VL 7B	×	0.00 (0.00)	**0.00 (0.00)**	0.00 (0.00)
Qwen3-VL 30B A3B	×	0.00 (0.00)	**0.00 (0.00)**	**0.04 (0.04)**
Qwen3-VL 8B	×	0.04 (0.04)	**0.00 (0.00)**	0.00 (0.00)

L Predicate Accuracy Results

We report the accuracy for individual predicates Q/A pairs for methods in the VLM-as-grounder class on ViPlan-BW in Tables 16 (No CoT) and 17 (CoT), and on ViPlan-HH in Tables 18 and 19 (CoT).

Some predicates are clearly harder than others to predict: in ViPlan-BW, the `clear` predicate tends to show much worse performance compared to the others, with some models even approaching random chance (0.50). `clear` indicates a block that can be moved, and is translated to "Is the block x the topmost of its column?", which is a harder question compared to the other predicates, which are more straight-forward. This is an example of what is sometimes known as a *derived predicate*, as it could also be obtained with a combination of `forall` and `on`, which confirms that predicates encoding higher-level relationships can be more challenging also for VLMs. The accuracy also tends to worsen as the splits increase in difficulty, which suggests that having more objects in the image (as harder splits have more blocks) results in a harder task for the VLM.

The performance is overall worse for ViPlan-HH, where many predicates, such as `nextto`, `open` and `reachable` fail to reach 90% accuracy even for the best-performing models. This can be attributed to the ambiguity of the domain: if in ViPlan-BW all predicates are distinctly and correctly identifiable, asking if an object is reachable by the agent or next to another object can require a degree of interpretation, which is challenging for VLMs. More well-defined predicates, such as `holding` and `inside` show a higher accuracy, confirming this hypothesis.

Furthermore, our experiments also measure the individual performance by ground truth answer (yes-no), which we omit from this manuscript for brevity. This reveals that some models, like Molmo and the AyaVision family, have a strong bias towards answering "yes", while others like DeepSeek-VL2 tend to always answer "no", leading to poor performance.

Table 16: **Individual Predicate Accuracy for Ground on ViPlan-BW.** The table shows the accuracy for each predicate in each split. Bolded values show the best accuracy for each predicate and split. Standard error of the mean is reported in parenthesis.

Model	Split	clear	incolumn	leftof	on	rightof
AyaVision 32B	Simple	0.70 (0.03)	0.77 (0.02)	**1.00 (0.00)**	0.69 (0.02)	**1.00 (0.00)**
	Medium	0.74 (0.02)	0.76 (0.01)	0.98 (0.00)	0.57 (0.01)	**1.00 (0.00)**
	Hard	0.59 (0.02)	0.79 (0.01)	**1.00 (0.00)**	0.55 (0.01)	**1.00 (0.00)**
AyaVision 8B	Simple	0.78 (0.02)	0.72 (0.01)	0.70 (0.01)	0.91 (0.01)	0.48 (0.01)
	Medium	0.62 (0.04)	0.75 (0.01)	0.62 (0.02)	0.89 (0.01)	0.53 (0.02)
	Hard	0.57 (0.03)	0.81 (0.01)	0.66 (0.02)	0.91 (0.01)	0.53 (0.02)
Cosmos-Reason2 8B	Simple	0.96 (0.02)	0.96 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	0.83 (0.02)	0.98 (0.00)	**1.00 (0.00)**	0.93 (0.00)	**1.00 (0.00)**
	Hard	0.85 (0.02)	0.97 (0.00)	**1.00 (0.00)**	0.93 (0.00)	**1.00 (0.00)**
DeepSeek-VL2	Simple	0.23 (0.05)	0.95 (0.01)	0.92 (0.01)	0.93 (0.02)	**1.00 (0.00)**
	Medium	0.33 (0.04)	0.95 (0.01)	0.88 (0.01)	0.81 (0.02)	0.99 (0.00)
	Hard	0.54 (0.04)	0.98 (0.01)	0.88 (0.02)	0.82 (0.01)	**1.00 (0.00)**
GPT-4.1	Simple	0.99 (0.01)	0.99 (0.01)	**1.00 (0.00)**	0.99 (0.01)	**1.00 (0.00)**
	Medium	0.93 (0.02)	0.98 (0.00)	**1.00 (0.00)**	0.98 (0.00)	**1.00 (0.00)**
	Hard	0.95 (0.01)	0.97 (0.00)	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
GPT-4.1 Nano	Simple	0.67 (0.05)	0.97 (0.01)	0.90 (0.01)	0.91 (0.02)	0.86 (0.02)
	Medium	0.70 (0.02)	0.91 (0.01)	0.79 (0.01)	0.87 (0.01)	0.86 (0.01)
	Hard	0.54 (0.02)	0.94 (0.01)	0.89 (0.01)	0.79 (0.01)	0.86 (0.01)
GPT-5.2	Simple	0.98 (0.02)	0.98 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	0.95 (0.02)	0.98 (0.00)	**1.00 (0.00)**	0.98 (0.00)	**1.00 (0.00)**
	Hard	0.95 (0.01)	0.97 (0.00)	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
Gemma-3 12B	Simple	0.57 (0.06)	0.94 (0.01)	**1.00 (0.00)**	0.96 (0.01)	**1.00 (0.00)**
	Medium	0.64 (0.04)	0.94 (0.01)	**1.00 (0.00)**	0.93 (0.01)	**1.00 (0.00)**
	Hard	0.69 (0.03)	0.91 (0.01)	**1.00 (0.00)**	0.88 (0.01)	**1.00 (0.00)**
Gemma-3 27B	Simple	0.85 (0.03)	0.96 (0.01)	0.95 (0.01)	0.96 (0.01)	**1.00 (0.00)**
	Medium	0.62 (0.04)	0.96 (0.01)	0.93 (0.01)	0.92 (0.01)	**1.00 (0.00)**
	Hard	0.69 (0.04)	0.94 (0.01)	0.97 (0.01)	0.90 (0.01)	**1.00 (0.00)**
InternVL3 78B	Simple	**1.00 (0.00)**	0.99 (0.01)	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**
	Medium	0.99 (0.01)	**1.00 (0.00)**	**1.00 (0.00)**	0.98 (0.01)	0.98 (0.01)
	Hard	0.96 (0.01)	0.99 (0.00)	**1.00 (0.00)**	0.96 (0.00)	0.99 (0.00)
InternVL3 8B	Simple	0.73 (0.03)	0.97 (0.01)	**1.00 (0.00)**	0.93 (0.01)	**1.00 (0.00)**
	Medium	0.66 (0.03)	0.95 (0.01)	**1.00 (0.00)**	0.92 (0.01)	**1.00 (0.00)**
	Hard	0.58 (0.03)	0.97 (0.00)	**1.00 (0.00)**	0.88 (0.01)	0.99 (0.00)
InternVL3.5 30B A3B	Simple	0.90 (0.03)	0.99 (0.00)	**1.00 (0.00)**	0.99 (0.01)	**1.00 (0.00)**
	Medium	0.79 (0.03)	0.98 (0.00)	**1.00 (0.00)**	0.98 (0.00)	**1.00 (0.00)**
	Hard	0.88 (0.02)	0.98 (0.00)	**1.00 (0.00)**	0.96 (0.00)	**1.00 (0.00)**
InternVL3.5 38B	Simple	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**
	Medium	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**	0.95 (0.01)	**1.00 (0.00)**
	Hard	**1.00 (0.00)**	0.99 (0.00)	**1.00 (0.00)**	0.95 (0.00)	**1.00 (0.00)**
InternVL3.5 8B	Simple	0.93 (0.02)	0.99 (0.00)	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**
	Medium	0.82 (0.02)	**1.00 (0.00)**	**1.00 (0.00)**	**0.99 (0.00)**	**1.00 (0.00)**
	Hard	0.81 (0.02)	**1.00 (0.00)**	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
LLaVA-Onevision 72B	Simple	0.49 (0.06)	0.94 (0.01)	**1.00 (0.00)**	0.95 (0.01)	**1.00 (0.00)**
	Medium	0.47 (0.05)	0.91 (0.01)	**1.00 (0.00)**	0.93 (0.01)	**1.00 (0.00)**
	Hard	0.68 (0.04)	0.94 (0.01)	**1.00 (0.00)**	0.91 (0.01)	**1.00 (0.00)**
LLaVA-Onevision 7B	Simple	0.40 (0.06)	0.90 (0.02)	0.97 (0.01)	0.96 (0.01)	0.95 (0.01)
	Medium	0.36 (0.04)	0.90 (0.01)	0.96 (0.01)	0.83 (0.01)	0.95 (0.01)
	Hard	0.52 (0.03)	0.94 (0.01)	0.96 (0.01)	0.79 (0.01)	0.93 (0.01)
Mistral-Small-3.1 24B	Simple	0.89 (0.03)	0.98 (0.01)	**1.00 (0.00)**	0.99 (0.01)	**1.00 (0.00)**
	Medium	0.72 (0.04)	0.96 (0.01)	0.98 (0.01)	0.95 (0.01)	0.99 (0.00)
	Hard	0.75 (0.03)	0.98 (0.01)	**1.00 (0.00)**	0.94 (0.01)	0.99 (0.00)
Molmo 7B	Simple	0.77 (0.02)	0.98 (0.00)	0.95 (0.01)	0.88 (0.01)	0.97 (0.00)
	Medium	0.81 (0.01)	0.97 (0.00)	0.96 (0.00)	0.86 (0.01)	0.97 (0.00)
	Hard	0.77 (0.01)	0.95 (0.00)	0.96 (0.00)	0.88 (0.00)	0.92 (0.00)
Phi-4 Multimodal	Simple	0.44 (0.05)	0.96 (0.01)	**1.00 (0.00)**	0.77 (0.02)	**1.00 (0.00)**
	Medium	0.36 (0.04)	0.97 (0.01)	**1.00 (0.00)**	0.71 (0.02)	**1.00 (0.00)**
	Hard	0.54 (0.03)	0.95 (0.01)	**1.00 (0.00)**	0.64 (0.01)	**1.00 (0.00)**
Qwen2.5-VL 72B	Simple	0.95 (0.02)	0.95 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	0.97 (0.01)	0.97 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Hard	0.94 (0.02)	0.96 (0.01)	**1.00 (0.00)**	0.96 (0.01)	**1.00 (0.00)**
Qwen2.5-VL 7B	Simple	0.95 (0.02)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**	0.99 (0.00)
	Medium	0.87 (0.03)	0.98 (0.00)	0.98 (0.00)	0.98 (0.00)	0.97 (0.01)
	Hard	0.83 (0.03)	0.97 (0.01)	0.90 (0.01)	0.96 (0.01)	0.94 (0.01)
Qwen3-VL 30B A3B	Simple	0.91 (0.02)	0.95 (0.01)	0.99 (0.00)	0.99 (0.00)	**1.00 (0.00)**
	Medium	0.74 (0.03)	0.97 (0.01)	0.99 (0.00)	0.97 (0.01)	**1.00 (0.00)**
	Hard	0.85 (0.02)	0.97 (0.01)	**1.00 (0.00)**	0.95 (0.01)	**1.00 (0.00)**
Qwen3-VL 32B	Simple	0.94 (0.02)	0.96 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	0.89 (0.02)	0.97 (0.00)	**1.00 (0.00)**	0.97 (0.00)	**1.00 (0.00)**
	Hard	0.86 (0.02)	0.96 (0.00)	**1.00 (0.00)**	0.95 (0.00)	**1.00 (0.00)**
Qwen3-VL 8B	Simple	0.93 (0.03)	0.98 (0.01)	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**
	Medium	0.84 (0.02)	0.98 (0.00)	**1.00 (0.00)**	0.98 (0.00)	**1.00 (0.00)**
	Hard	0.89 (0.02)	0.99 (0.00)	**1.00 (0.00)**	0.97 (0.00)	**1.00 (0.00)**

Table 17: **Individual Predicate Accuracy for Ground + CoT on ViPlan-BW.** The table shows the accuracy for each predicate in each split. Bolded values show the best accuracy for each predicate and split. Standard error of the mean is reported in parenthesis.

Model	Split	clear	incolumn	leftof	on	rightof
AyaVision 32B	Simple	0.73 (0.03)	0.78 (0.02)	**1.00 (0.00)**	0.74 (0.02)	0.98 (0.00)
	Medium	0.69 (0.02)	0.80 (0.01)	**1.00 (0.00)**	0.66 (0.01)	0.98 (0.00)
	Hard	0.53 (0.02)	0.84 (0.01)	**1.00 (0.00)**	0.65 (0.01)	0.96 (0.01)
AyaVision 8B	Simple	0.52 (0.06)	0.58 (0.03)	0.46 (0.02)	0.85 (0.02)	0.30 (0.02)
	Medium	0.45 (0.05)	0.58 (0.02)	0.44 (0.02)	0.80 (0.02)	0.33 (0.02)
	Hard	0.53 (0.04)	0.65 (0.02)	0.43 (0.02)	0.76 (0.01)	0.34 (0.02)
Cosmos-Reason2 8B	Simple	0.93 (0.02)	0.91 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	0.86 (0.02)	0.96 (0.00)	**1.00 (0.00)**	0.96 (0.00)	**1.00 (0.00)**
	Hard	0.92 (0.01)	0.92 (0.01)	**1.00 (0.00)**	0.95 (0.00)	**1.00 (0.00)**
DeepSeek-VL2	Simple	0.71 (0.03)	0.96 (0.01)	0.92 (0.01)	0.77 (0.02)	0.96 (0.01)
	Medium	0.64 (0.03)	0.87 (0.01)	0.91 (0.01)	0.74 (0.01)	0.94 (0.01)
	Hard	0.60 (0.03)	0.95 (0.01)	0.94 (0.01)	0.76 (0.01)	0.96 (0.01)
GPT-4.1	Simple	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**	0.99 (0.01)	**1.00 (0.00)**
	Medium	0.95 (0.01)	0.97 (0.00)	**1.00 (0.00)**	0.97 (0.00)	**1.00 (0.00)**
	Hard	0.96 (0.01)	0.96 (0.00)	**1.00 (0.00)**	**0.97 (0.00)**	**1.00 (0.00)**
GPT-4.1 Nano	Simple	0.88 (0.03)	0.91 (0.01)	**1.00 (0.00)**	0.91 (0.01)	**1.00 (0.00)**
	Medium	0.84 (0.02)	0.93 (0.01)	**1.00 (0.00)**	0.94 (0.01)	**1.00 (0.00)**
	Hard	0.84 (0.02)	0.92 (0.01)	**1.00 (0.00)**	0.88 (0.01)	**1.00 (0.00)**
Gemma-3 12B	Simple	0.83 (0.03)	0.88 (0.01)	0.97 (0.01)	0.84 (0.02)	0.99 (0.00)
	Medium	0.68 (0.04)	0.92 (0.01)	0.95 (0.01)	0.81 (0.02)	0.98 (0.01)
	Hard	0.72 (0.03)	0.91 (0.01)	0.94 (0.01)	0.81 (0.01)	0.99 (0.00)
Gemma-3 27B	Simple	0.86 (0.03)	0.94 (0.01)	**1.00 (0.00)**	0.89 (0.02)	**1.00 (0.00)**
	Medium	0.80 (0.02)	0.95 (0.01)	0.99 (0.00)	0.83 (0.01)	**1.00 (0.00)**
	Hard	0.84 (0.02)	0.94 (0.01)	**1.00 (0.00)**	0.84 (0.01)	**1.00 (0.00)**
InternVL3 78B	Simple	0.98 (0.02)	0.99 (0.01)	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**
	Medium	**1.00 (0.00)**	0.98 (0.00)	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
	Hard	**0.99 (0.01)**	**0.98 (0.00)**	**1.00 (0.00)**	0.95 (0.00)	**1.00 (0.00)**
InternVL3 8B	Simple	0.77 (0.04)	0.98 (0.01)	0.99 (0.00)	0.82 (0.02)	0.99 (0.00)
	Medium	0.73 (0.03)	0.95 (0.01)	0.99 (0.00)	0.80 (0.01)	0.99 (0.00)
	Hard	0.61 (0.03)	0.96 (0.01)	**1.00 (0.00)**	0.71 (0.01)	**1.00 (0.00)**
InternVL3.5 30B A3B	Simple	0.91 (0.03)	0.99 (0.01)	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**
	Medium	0.85 (0.02)	0.98 (0.00)	**1.00 (0.00)**	0.94 (0.01)	**1.00 (0.00)**
	Hard	0.76 (0.02)	0.96 (0.01)	**1.00 (0.00)**	0.90 (0.01)	**1.00 (0.00)**
InternVL3.5 38B	Simple	**1.00 (0.00)**	**1.00 (0.00)**	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	**1.00 (0.00)**	**0.99 (0.00)**	**1.00 (0.00)**	0.94 (0.01)	**1.00 (0.00)**
	Hard	0.98 (0.01)	0.97 (0.00)	**1.00 (0.00)**	0.89 (0.01)	**1.00 (0.00)**
InternVL3.5 8B	Simple	0.98 (0.01)	0.97 (0.01)	**1.00 (0.00)**	0.98 (0.01)	**1.00 (0.00)**
	Medium	0.88 (0.02)	0.97 (0.00)	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
	Hard	0.87 (0.01)	**0.98 (0.00)**	**1.00 (0.00)**	0.96 (0.00)	**1.00 (0.00)**
LLaVA-Onevision 72B	Simple	0.76 (0.04)	0.92 (0.01)	**1.00 (0.00)**	0.93 (0.01)	**1.00 (0.00)**
	Medium	0.54 (0.04)	0.92 (0.01)	0.99 (0.00)	0.95 (0.01)	**1.00 (0.00)**
	Hard	0.73 (0.04)	0.93 (0.01)	**1.00 (0.00)**	0.93 (0.01)	**1.00 (0.00)**
LLaVA-Onevision 7B	Simple	0.42 (0.06)	0.89 (0.02)	**1.00 (0.00)**	0.93 (0.02)	**1.00 (0.00)**
	Medium	0.41 (0.04)	0.82 (0.02)	0.96 (0.01)	0.84 (0.01)	0.98 (0.01)
	Hard	0.63 (0.04)	0.92 (0.01)	0.99 (0.00)	0.83 (0.01)	0.98 (0.01)
Mistral-Small-3.1 24B	Simple	0.96 (0.02)	0.96 (0.01)	**1.00 (0.00)**	0.95 (0.01)	**1.00 (0.00)**
	Medium	0.90 (0.02)	0.95 (0.01)	**1.00 (0.00)**	0.88 (0.01)	**1.00 (0.00)**
	Hard	0.87 (0.02)	0.97 (0.01)	**1.00 (0.00)**	0.87 (0.01)	**1.00 (0.00)**
Molmo 7B	Simple	0.28 (0.05)	0.83 (0.02)	0.78 (0.02)	0.93 (0.02)	0.71 (0.02)
	Medium	0.37 (0.04)	0.87 (0.01)	0.76 (0.02)	0.80 (0.02)	0.64 (0.02)
	Hard	0.49 (0.04)	0.80 (0.02)	0.69 (0.02)	0.81 (0.01)	0.61 (0.02)
Phi-4 Multimodal	Simple	0.09 (0.03)	0.12 (0.02)	0.04 (0.01)	0.19 (0.03)	0.04 (0.01)
	Medium	0.12 (0.03)	0.29 (0.02)	0.03 (0.01)	0.26 (0.02)	0.05 (0.01)
	Hard	0.11 (0.03)	0.35 (0.02)	0.06 (0.01)	0.22 (0.01)	0.05 (0.01)
Qwen2.5-VL 72B	Simple	0.92 (0.03)	0.92 (0.01)	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**
	Medium	0.95 (0.02)	0.98 (0.00)	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
	Hard	0.93 (0.02)	0.94 (0.01)	**1.00 (0.00)**	0.94 (0.01)	**1.00 (0.00)**
Qwen2.5-VL 7B	Simple	0.78 (0.04)	0.98 (0.01)	**1.00 (0.00)**	0.95 (0.01)	**1.00 (0.00)**
	Medium	0.72 (0.03)	0.95 (0.01)	**1.00 (0.00)**	0.77 (0.01)	**1.00 (0.00)**
	Hard	0.76 (0.02)	0.97 (0.00)	**1.00 (0.00)**	0.77 (0.01)	**1.00 (0.00)**
Qwen3-VL 30B A3B	Simple	0.96 (0.02)	0.91 (0.01)	**1.00 (0.00)**	0.96 (0.01)	**1.00 (0.00)**
	Medium	0.93 (0.01)	0.93 (0.01)	**1.00 (0.00)**	**0.98 (0.00)**	**1.00 (0.00)**
	Hard	0.95 (0.01)	0.92 (0.01)	**1.00 (0.00)**	**0.97 (0.00)**	**1.00 (0.00)**
Qwen3-VL 32B	Simple	0.90 (0.03)	0.93 (0.01)	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**
	Medium	0.93 (0.02)	0.96 (0.01)	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**
	Hard	0.92 (0.02)	0.94 (0.01)	**1.00 (0.00)**	0.96 (0.01)	**1.00 (0.00)**
Qwen3-VL 8B	Simple	0.98 (0.01)	0.97 (0.01)	**1.00 (0.00)**	0.97 (0.01)	**1.00 (0.00)**
	Medium	0.97 (0.01)	0.95 (0.01)	**1.00 (0.00)**	0.96 (0.00)	**1.00 (0.00)**
	Hard	0.96 (0.01)	0.95 (0.00)	**1.00 (0.00)**	0.94 (0.00)	**1.00 (0.00)**

Table 18: **Individual Predicate Accuracy for Ground on ViPlan-HH.** The table shows the accuracy for each predicate in each split. Bolded values show the best accuracy for each predicate and split. Standard error of the mean is reported in parenthesis.

Model	Split	holding	inside	nextto	ontop	open	reachable
AyaVision 32B	Simple	0.79 (0.04)	0.79 (0.04)	0.34 (0.03)	0.78 (0.03)	0.30 (0.05)	0.71 (0.03)
	Medium	0.86 (0.03)	0.76 (0.03)	0.40 (0.01)	0.87 (0.01)	0.47 (0.07)	0.64 (0.03)
	Hard	0.92 (0.02)	0.57 (0.03)	0.55 (0.02)	0.78 (0.01)	0.49 (0.04)	0.58 (0.02)
AyaVision 8B	Simple	0.59 (0.05)	0.61 (0.05)	0.12 (0.02)	0.57 (0.03)	0.39 (0.06)	0.68 (0.04)
	Medium	0.64 (0.03)	0.52 (0.03)	0.14 (0.01)	0.67 (0.01)	0.33 (0.05)	0.63 (0.03)
	Hard	0.57 (0.04)	0.42 (0.03)	0.19 (0.02)	0.63 (0.02)	0.79 (0.04)	0.38 (0.03)
Cosmos-Reason2 8B	Simple	0.75 (0.04)	0.63 (0.05)	0.34 (0.03)	0.61 (0.03)	0.51 (0.05)	0.76 (0.03)
	Medium	0.78 (0.02)	0.88 (0.02)	0.41 (0.01)	0.83 (0.01)	**0.79 (0.05)**	0.65 (0.02)
	Hard	0.85 (0.03)	0.71 (0.03)	0.29 (0.02)	0.55 (0.02)	0.76 (0.04)	0.47 (0.03)
DeepSeek-VL2	Simple	0.91 (0.02)	0.81 (0.03)	0.54 (0.02)	0.85 (0.02)	0.46 (0.05)	0.69 (0.03)
	Medium	0.98 (0.00)	0.90 (0.02)	0.59 (0.00)	0.92 (0.00)	0.77 (0.04)	0.50 (0.01)
	Hard	0.91 (0.01)	0.93 (0.01)	0.45 (0.01)	0.89 (0.01)	**0.88 (0.02)**	**0.83 (0.01)**
GPT-4.1	Simple	0.79 (0.03)	0.80 (0.03)	0.52 (0.02)	0.83 (0.02)	0.50 (0.04)	0.66 (0.03)
	Medium	0.81 (0.02)	0.94 (0.02)	0.55 (0.01)	0.89 (0.01)	0.74 (0.04)	0.72 (0.02)
	Hard	0.86 (0.02)	0.79 (0.03)	0.56 (0.02)	0.86 (0.01)	0.77 (0.04)	0.49 (0.03)
GPT-4.1 Nano	Simple	0.85 (0.04)	0.66 (0.05)	0.18 (0.02)	0.73 (0.02)	0.49 (0.06)	0.67 (0.04)
	Medium	0.89 (0.02)	0.83 (0.03)	0.30 (0.01)	0.84 (0.01)	0.62 (0.06)	0.55 (0.03)
	Hard	0.86 (0.04)	0.74 (0.03)	0.21 (0.02)	0.69 (0.02)	0.64 (0.06)	0.53 (0.04)
GPT-5.2	Simple	0.86 (0.03)	0.67 (0.04)	0.52 (0.02)	0.84 (0.02)	0.59 (0.04)	0.49 (0.03)
	Medium	0.95 (0.01)	0.94 (0.01)	0.64 (0.01)	0.94 (0.00)	0.53 (0.03)	0.42 (0.01)
	Hard	0.98 (0.00)	0.88 (0.01)	0.73 (0.00)	**0.94 (0.00)**	0.53 (0.02)	0.63 (0.01)
Gemma-3 12B	Simple	0.63 (0.05)	0.71 (0.07)	0.23 (0.03)	0.65 (0.03)	0.27 (0.05)	0.70 (0.04)
	Medium	0.71 (0.03)	0.79 (0.03)	0.61 (0.01)	0.76 (0.01)	0.44 (0.06)	0.64 (0.02)
	Hard	0.63 (0.04)	0.76 (0.03)	0.39 (0.02)	0.76 (0.02)	0.68 (0.05)	0.43 (0.03)
Gemma-3 27B	Simple	0.80 (0.03)	0.80 (0.04)	0.52 (0.03)	**0.92 (0.01)**	0.46 (0.05)	0.50 (0.03)
	Medium	0.86 (0.01)	0.95 (0.01)	0.63 (0.01)	0.94 (0.00)	0.54 (0.03)	0.55 (0.02)
	Hard	0.87 (0.01)	0.85 (0.01)	0.63 (0.01)	0.93 (0.00)	0.38 (0.03)	0.50 (0.01)
InternVL3 78B	Simple	0.89 (0.03)	0.82 (0.03)	0.59 (0.02)	0.77 (0.02)	0.38 (0.04)	0.76 (0.03)
	Medium	0.86 (0.02)	0.87 (0.02)	0.48 (0.01)	0.83 (0.01)	0.37 (0.04)	0.74 (0.02)
	Hard	0.83 (0.03)	0.83 (0.03)	0.43 (0.02)	0.83 (0.02)	0.79 (0.05)	0.42 (0.03)
InternVL3 8B	Simple	0.91 (0.02)	0.83 (0.02)	0.61 (0.02)	0.84 (0.01)	0.36 (0.04)	0.76 (0.02)
	Medium	0.91 (0.01)	0.89 (0.01)	0.65 (0.01)	0.91 (0.01)	0.55 (0.03)	0.71 (0.02)
	Hard	0.90 (0.02)	0.93 (0.02)	0.71 (0.01)	0.90 (0.01)	0.62 (0.05)	0.44 (0.02)
InternVL3.5 30B A3B	Simple	0.93 (0.03)	0.87 (0.03)	0.58 (0.03)	0.76 (0.03)	0.39 (0.05)	0.63 (0.03)
	Medium	0.92 (0.01)	0.90 (0.02)	0.50 (0.01)	0.92 (0.01)	0.56 (0.04)	0.67 (0.02)
	Hard	0.87 (0.02)	0.59 (0.03)	0.58 (0.01)	0.81 (0.01)	0.44 (0.04)	0.51 (0.02)
InternVL3.5 38B	Simple	0.86 (0.03)	0.56 (0.06)	0.32 (0.03)	0.55 (0.03)	0.40 (0.04)	0.60 (0.03)
	Medium	0.86 (0.02)	0.78 (0.02)	0.30 (0.01)	0.64 (0.01)	0.68 (0.05)	0.68 (0.02)
	Hard	0.66 (0.03)	0.61 (0.04)	0.29 (0.01)	0.54 (0.01)	0.74 (0.05)	0.63 (0.03)
InternVL3.5 8B	Simple	0.81 (0.04)	0.73 (0.04)	0.46 (0.03)	0.87 (0.02)	0.53 (0.06)	**0.79 (0.03)**
	Medium	0.91 (0.02)	0.95 (0.01)	0.48 (0.01)	0.91 (0.01)	0.73 (0.04)	0.67 (0.02)
	Hard	0.94 (0.01)	0.84 (0.03)	0.39 (0.01)	0.92 (0.01)	0.43 (0.05)	0.71 (0.02)
LLaVA-Onevision 72B	Simple	0.87 (0.04)	0.48 (0.09)	0.50 (0.04)	0.84 (0.03)	0.20 (0.06)	0.64 (0.04)
	Medium	0.94 (0.01)	0.91 (0.01)	0.68 (0.01)	0.94 (0.01)	0.60 (0.03)	0.63 (0.02)
	Hard	0.96 (0.01)	0.81 (0.03)	0.59 (0.01)	0.88 (0.01)	0.60 (0.06)	0.68 (0.03)
LLaVA-Onevision 7B	Simple	0.87 (0.03)	0.82 (0.03)	0.28 (0.02)	0.81 (0.02)	0.64 (0.04)	0.44 (0.03)
	Medium	0.89 (0.02)	0.96 (0.01)	0.41 (0.01)	0.88 (0.01)	0.73 (0.03)	0.52 (0.02)
	Hard	0.94 (0.01)	0.88 (0.03)	0.45 (0.01)	**0.94 (0.01)**	0.84 (0.04)	0.45 (0.03)
Mistral-Small-3.1 24B	Simple	**0.98 (0.01)**	0.84 (0.04)	0.72 (0.02)	0.87 (0.01)	0.67 (0.03)	0.28 (0.02)
	Medium	**1.00 (0.00)**	**1.00 (0.00)**	0.79 (0.01)	0.94 (0.00)	0.73 (0.02)	0.40 (0.01)
	Hard	**0.99 (0.00)**	**0.96 (0.00)**	0.78 (0.00)	0.93 (0.00)	0.72 (0.01)	0.40 (0.01)
Molmo 7B	Simple	0.88 (0.03)	0.75 (0.04)	0.51 (0.03)	0.58 (0.03)	0.29 (0.05)	0.65 (0.04)
	Medium	0.86 (0.02)	0.60 (0.03)	0.55 (0.01)	0.68 (0.01)	0.54 (0.04)	**0.76 (0.02)**
	Hard	0.90 (0.01)	0.69 (0.01)	0.62 (0.00)	0.70 (0.00)	0.61 (0.03)	0.71 (0.01)
Phi-4 Multimodal	Simple	0.51 (0.06)	0.39 (0.05)	0.20 (0.02)	0.14 (0.02)	0.33 (0.06)	0.63 (0.04)
	Medium	0.69 (0.03)	0.38 (0.03)	0.11 (0.01)	0.18 (0.01)	0.26 (0.04)	0.57 (0.02)
	Hard	0.56 (0.03)	0.43 (0.03)	0.24 (0.01)	0.18 (0.01)	0.59 (0.04)	0.63 (0.03)
Qwen2.5-VL 72B	Simple	0.95 (0.01)	0.75 (0.05)	**0.79 (0.02)**	0.83 (0.02)	0.60 (0.04)	0.35 (0.02)
	Medium	0.95 (0.01)	0.88 (0.01)	**0.81 (0.01)**	0.93 (0.01)	0.51 (0.03)	0.36 (0.01)
	Hard	0.95 (0.01)	0.83 (0.01)	0.72 (0.00)	0.91 (0.00)	0.64 (0.02)	0.39 (0.01)
Qwen2.5-VL 7B	Simple	0.96 (0.01)	**0.89 (0.04)**	0.73 (0.02)	0.86 (0.02)	**0.72 (0.03)**	0.47 (0.02)
	Medium	0.91 (0.01)	0.92 (0.01)	0.77 (0.01)	**0.96 (0.00)**	0.44 (0.03)	0.43 (0.01)
	Hard	0.96 (0.01)	0.90 (0.01)	**0.80 (0.00)**	0.93 (0.00)	0.60 (0.02)	0.37 (0.01)
Qwen3-VL 30B A3B	Simple	0.92 (0.02)	0.80 (0.02)	0.71 (0.02)	0.89 (0.01)	0.35 (0.04)	0.64 (0.03)
	Medium	0.92 (0.01)	0.83 (0.02)	0.53 (0.01)	0.86 (0.01)	0.43 (0.03)	0.72 (0.02)
	Hard	0.90 (0.02)	0.82 (0.02)	0.59 (0.02)	0.82 (0.01)	0.76 (0.04)	0.51 (0.03)
Qwen3-VL 32B	Simple	0.91 (0.02)	0.73 (0.05)	0.56 (0.03)	0.89 (0.02)	0.52 (0.04)	0.67 (0.03)
	Medium	0.90 (0.01)	0.83 (0.02)	0.55 (0.01)	0.88 (0.01)	0.48 (0.04)	0.55 (0.02)
	Hard	0.90 (0.02)	0.82 (0.02)	0.43 (0.01)	0.83 (0.01)	0.71 (0.03)	0.65 (0.02)
Qwen3-VL 8B	Simple	0.87 (0.02)	0.85 (0.02)	0.46 (0.02)	0.83 (0.01)	0.27 (0.03)	0.62 (0.02)
	Medium	0.90 (0.01)	0.91 (0.01)	0.57 (0.01)	0.88 (0.01)	0.45 (0.04)	0.54 (0.02)
	Hard	0.91 (0.02)	0.88 (0.02)	0.52 (0.01)	0.87 (0.01)	0.64 (0.04)	0.72 (0.02)

Table 19: **Individual Predicate Accuracy for Ground + CoT on ViPlan-HH.** The table shows the accuracy for each predicate in each split. Bolded values show the best accuracy for each predicate and split. Standard error of the mean is reported in parenthesis.

Model	Split	holding	inside	nextto	ontop	open	reachable
AyaVision 32B	Simple	0.88 (0.03)	0.85 (0.03)	**0.66 (0.02)**	0.82 (0.02)	0.51 (0.04)	0.62 (0.03)
	Medium	0.86 (0.02)	0.93 (0.01)	0.58 (0.01)	0.88 (0.01)	**0.82 (0.03)**	0.74 (0.02)
	Hard	0.87 (0.02)	0.80 (0.02)	0.59 (0.01)	0.92 (0.01)	0.80 (0.03)	0.52 (0.02)
AyaVision 8B	Simple	0.62 (0.04)	0.70 (0.03)	0.47 (0.02)	**0.92 (0.01)**	0.47 (0.04)	0.67 (0.03)
	Medium	0.52 (0.03)	0.72 (0.03)	0.57 (0.01)	0.90 (0.01)	0.45 (0.05)	0.59 (0.02)
	Hard	0.57 (0.02)	0.85 (0.01)	0.49 (0.01)	0.89 (0.00)	0.70 (0.03)	0.68 (0.02)
Cosmos-Reason2 8B	Simple	0.89 (0.03)	0.73 (0.04)	0.60 (0.03)	0.87 (0.02)	0.57 (0.04)	0.51 (0.03)
	Medium	**0.98 (0.01)**	**0.98 (0.01)**	0.55 (0.02)	0.82 (0.01)	0.65 (0.03)	0.52 (0.02)
	Hard	0.79 (0.02)	0.67 (0.02)	0.73 (0.01)	0.91 (0.01)	0.68 (0.03)	0.46 (0.02)
DeepSeek-VL2	Simple	0.72 (0.05)	0.79 (0.06)	0.61 (0.03)	0.76 (0.03)	0.53 (0.06)	0.74 (0.03)
	Medium	0.67 (0.02)	0.97 (0.01)	0.51 (0.01)	0.83 (0.01)	0.61 (0.04)	0.80 (0.01)
	Hard	0.42 (0.03)	0.85 (0.02)	0.61 (0.01)	0.86 (0.01)	0.88 (0.02)	0.34 (0.02)
GPT-4.1	Simple	0.87 (0.03)	0.68 (0.04)	0.60 (0.02)	0.81 (0.02)	0.52 (0.05)	0.66 (0.03)
	Medium	0.85 (0.02)	0.91 (0.01)	0.51 (0.01)	0.87 (0.01)	0.70 (0.03)	0.69 (0.02)
	Hard	0.92 (0.01)	0.78 (0.02)	0.55 (0.01)	0.85 (0.00)	0.63 (0.04)	0.64 (0.02)
GPT-4.1 Nano	Simple	0.79 (0.04)	0.88 (0.03)	0.60 (0.03)	0.78 (0.02)	0.34 (0.05)	0.60 (0.03)
	Medium	0.82 (0.02)	0.92 (0.01)	0.57 (0.01)	0.85 (0.01)	0.55 (0.04)	0.65 (0.02)
	Hard	0.71 (0.03)	0.78 (0.02)	0.58 (0.01)	0.73 (0.01)	0.53 (0.03)	0.62 (0.02)
Gemma-3 12B	Simple	0.64 (0.05)	0.83 (0.04)	0.37 (0.03)	0.83 (0.02)	0.58 (0.05)	0.71 (0.03)
	Medium	0.55 (0.03)	0.97 (0.01)	0.38 (0.01)	0.69 (0.01)	0.70 (0.04)	0.59 (0.02)
	Hard	0.48 (0.03)	0.89 (0.01)	0.38 (0.01)	0.84 (0.01)	0.38 (0.03)	0.41 (0.02)
Gemma-3 27B	Simple	0.83 (0.03)	0.73 (0.03)	0.61 (0.02)	0.91 (0.01)	0.43 (0.04)	0.50 (0.02)
	Medium	0.64 (0.02)	0.97 (0.01)	0.39 (0.01)	0.89 (0.01)	0.69 (0.03)	**0.81 (0.01)**
	Hard	0.79 (0.02)	0.96 (0.01)	0.33 (0.01)	0.89 (0.00)	0.80 (0.03)	0.82 (0.01)
InternVL3 78B	Simple	0.87 (0.03)	0.73 (0.05)	0.50 (0.03)	0.75 (0.02)	0.41 (0.05)	0.72 (0.03)
	Medium	0.76 (0.02)	0.95 (0.01)	0.45 (0.01)	0.86 (0.01)	0.75 (0.04)	0.73 (0.02)
	Hard	0.95 (0.01)	0.94 (0.01)	0.40 (0.01)	0.86 (0.01)	0.70 (0.04)	**0.92 (0.01)**
InternVL3 8B	Simple	0.73 (0.04)	0.76 (0.04)	0.53 (0.02)	0.67 (0.02)	0.36 (0.05)	0.68 (0.03)
	Medium	0.67 (0.02)	0.85 (0.02)	0.45 (0.01)	0.65 (0.01)	0.36 (0.04)	0.75 (0.02)
	Hard	0.82 (0.02)	0.52 (0.02)	0.40 (0.01)	0.61 (0.01)	0.67 (0.04)	0.75 (0.02)
InternVL3.5 30B A3B	Simple	0.94 (0.02)	0.66 (0.04)	0.60 (0.02)	0.88 (0.01)	0.59 (0.04)	0.58 (0.03)
	Medium	0.92 (0.01)	0.97 (0.01)	0.62 (0.01)	0.85 (0.01)	0.72 (0.04)	0.64 (0.02)
	Hard	0.90 (0.02)	0.87 (0.02)	0.71 (0.02)	0.86 (0.01)	0.61 (0.05)	0.49 (0.03)
InternVL3.5 38B	Simple	0.86 (0.03)	0.78 (0.03)	0.52 (0.02)	0.79 (0.02)	0.47 (0.04)	0.71 (0.03)
	Medium	0.76 (0.02)	0.79 (0.02)	0.47 (0.01)	0.80 (0.01)	0.50 (0.04)	0.69 (0.02)
	Hard	0.85 (0.01)	0.86 (0.01)	0.55 (0.01)	0.72 (0.01)	0.74 (0.02)	0.78 (0.01)
InternVL3.5 8B	Simple	0.75 (0.04)	0.84 (0.03)	0.39 (0.03)	0.80 (0.02)	0.55 (0.05)	0.73 (0.03)
	Medium	0.70 (0.03)	0.82 (0.02)	0.50 (0.01)	0.89 (0.01)	0.56 (0.04)	0.71 (0.02)
	Hard	0.72 (0.02)	0.89 (0.01)	0.61 (0.01)	0.87 (0.01)	0.62 (0.03)	0.35 (0.02)
LLaVA-Onevision 72B	Simple	0.86 (0.04)	0.50 (0.13)	**0.66 (0.04)**	0.87 (0.03)	0.27 (0.06)	0.59 (0.04)
	Medium	0.95 (0.01)	0.94 (0.01)	0.63 (0.01)	**0.94 (0.01)**	0.61 (0.04)	0.67 (0.02)
	Hard	0.89 (0.07)	0.61 (0.09)	**0.76 (0.04)**	0.92 (0.03)	0.53 (0.13)	0.69 (0.07)
LLaVA-Onevision 7B	Simple	0.91 (0.03)	0.74 (0.05)	0.32 (0.03)	0.77 (0.02)	0.55 (0.05)	0.45 (0.03)
	Medium	0.89 (0.02)	0.94 (0.02)	0.38 (0.01)	0.87 (0.01)	0.69 (0.04)	0.53 (0.02)
	Hard	0.95 (0.01)	0.92 (0.02)	0.45 (0.01)	**0.93 (0.01)**	0.79 (0.04)	0.49 (0.02)
Mistral-Small-3.1 24B	Simple	0.77 (0.04)	0.67 (0.05)	0.44 (0.03)	0.69 (0.03)	0.43 (0.05)	0.65 (0.03)
	Medium	0.59 (0.03)	0.89 (0.02)	0.46 (0.01)	0.79 (0.01)	0.40 (0.05)	0.73 (0.02)
	Hard	0.70 (0.04)	0.84 (0.02)	0.59 (0.02)	0.79 (0.02)	0.81 (0.03)	0.40 (0.03)
Molmo 7B	Simple	**0.99 (0.01)**	**0.95 (0.04)**	0.49 (0.02)	0.80 (0.02)	0.45 (0.06)	0.63 (0.03)
	Medium	0.72 (0.02)	0.92 (0.02)	0.56 (0.01)	0.87 (0.01)	0.77 (0.04)	0.59 (0.02)
	Hard	0.96 (0.01)	0.90 (0.01)	0.75 (0.01)	0.87 (0.01)	0.36 (0.02)	0.56 (0.01)
Phi-4 Multimodal	Simple	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)	0.00 (0.00)	**0.67 (0.19)**	0.00 (0.00)
	Medium	0.00 (0.00)	0.17 (0.11)	0.01 (0.00)	0.02 (0.01)	0.67 (0.19)	0.10 (0.04)
	Hard	0.22 (0.08)	0.26 (0.08)	0.02 (0.01)	0.27 (0.05)	0.64 (0.14)	0.05 (0.03)
Qwen2.5-VL 72B	Simple	0.90 (0.02)	0.82 (0.03)	0.65 (0.02)	0.80 (0.02)	0.53 (0.04)	0.54 (0.02)
	Medium	0.87 (0.02)	0.95 (0.01)	**0.66 (0.01)**	0.86 (0.01)	0.59 (0.04)	0.58 (0.02)
	Hard	0.92 (0.01)	**0.97 (0.01)**	0.66 (0.01)	0.82 (0.01)	0.76 (0.02)	0.68 (0.01)
Qwen2.5-VL 7B	Simple	0.84 (0.03)	0.84 (0.03)	0.55 (0.03)	0.83 (0.02)	0.49 (0.04)	0.61 (0.03)
	Medium	0.72 (0.02)	**0.98 (0.00)**	0.52 (0.01)	0.90 (0.00)	0.67 (0.03)	0.65 (0.01)
	Hard	0.61 (0.01)	0.96 (0.00)	0.46 (0.00)	0.75 (0.00)	0.45 (0.02)	0.58 (0.01)
Qwen3-VL 30B A3B	Simple	0.92 (0.02)	0.68 (0.04)	0.53 (0.02)	0.88 (0.02)	0.37 (0.04)	0.60 (0.03)
	Medium	0.90 (0.01)	0.78 (0.02)	0.58 (0.01)	0.87 (0.01)	0.56 (0.05)	0.69 (0.02)
	Hard	0.94 (0.01)	0.89 (0.01)	0.72 (0.01)	0.75 (0.01)	0.75 (0.02)	0.44 (0.01)
Qwen3-VL 32B	Simple	0.77 (0.04)	0.77 (0.04)	0.56 (0.03)	0.74 (0.02)	0.48 (0.05)	0.69 (0.03)
	Medium	0.91 (0.01)	0.90 (0.01)	0.52 (0.01)	0.84 (0.01)	0.60 (0.04)	0.76 (0.02)
	Hard	0.92 (0.01)	0.92 (0.01)	0.55 (0.01)	0.77 (0.01)	0.73 (0.03)	0.64 (0.02)
Qwen3-VL 8B	Simple	0.85 (0.03)	0.92 (0.02)	0.64 (0.02)	0.82 (0.02)	0.52 (0.03)	**0.84 (0.02)**
	Medium	0.94 (0.01)	0.89 (0.01)	0.61 (0.01)	0.73 (0.01)	0.65 (0.03)	0.77 (0.01)
	Hard	**0.97 (0.01)**	0.96 (0.01)	0.57 (0.01)	0.85 (0.01)	**0.92 (0.01)**	0.51 (0.01)